

Aprendizaje automático no supervisado

Guillermo Torralba Elipe

Tema 5

Clustering basado en densidad DBSCAN

Índice

- Introducción
- Tipos de agrupamiento
- Algoritmos basados en densidad
- DBSCAN
- Hiperparámetros
- Evaluación de agrupaciones
- Ventajas/desventajas de DBSCAN
- Variaciones de DBSCAN

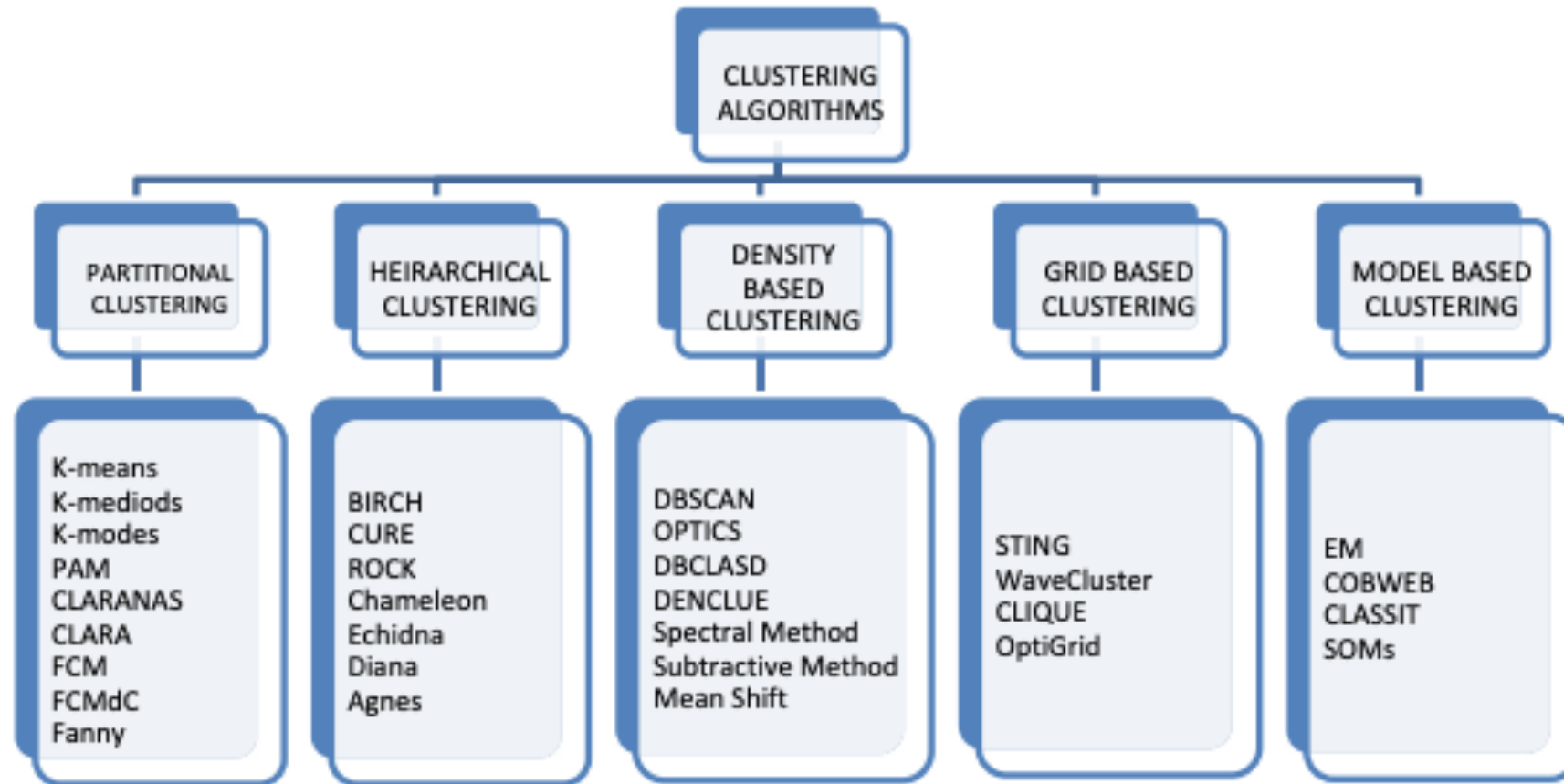
¿Cuáles son las diferencias entre k -means y el agrupamiento jerárquico?

¿Cuáles son sus limitaciones?

Introducción

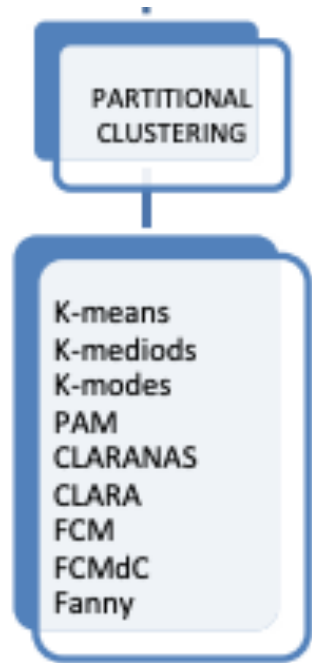
- Hasta ahora hemos visto técnicas de agrupamiento que funcionan bien con grupos esféricos (más para k-Means y menos para el jerárquico).
- Ambos métodos se ven afectados por la presencia de ruido y/o valores atípicos.
- DBSCAN es una técnica de agrupamiento basada en la densidad, distinto de los vistos hasta ahora (basados en la distancia)
- DBSCAN no necesita conocer previamente el número de grupos, no necesita conjuntos convexos ni esféricos y, además, es menos sensible al ruido (incluso puede identificarlo).

Tipos de agrupamiento



Ghosal, A., Nandy, A., Das, A.K., Goswami, S., & Panday, M. (2019). A Short Review on Different Clustering Techniques and Their Applications. Advances in Intelligent Systems and Computing.

Basados en particiones



El conjunto de datos se divide en grupos.

Todos los datos deben pertenecer a algún grupo.

Se necesita conocer el número de grupos previamente.

Minimización de una función de pérdida que depende de la distancia a un centroide.

Jerárquico

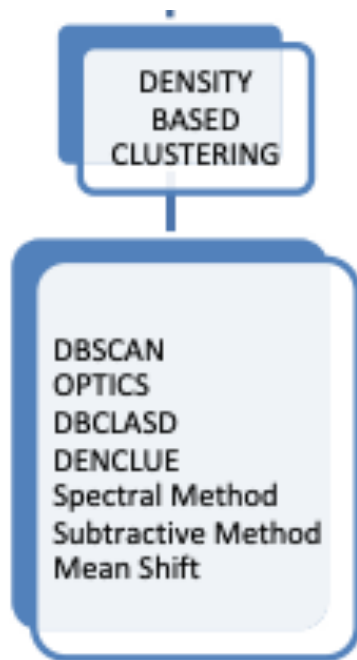


Dos tipos de enfoques: top-down y bottom-up.

No se necesita conocer el número de grupos de antemano.

El número de grupos se obtiene después mediante el uso de un dendrograma.

Basados en densidad (*)



Identifican regiones de densidades distintas.

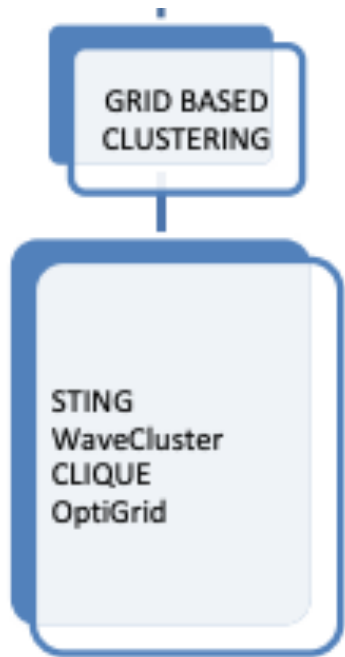
Las regiones de alta densidad se caracterizan como grupos mientras que las de baja densidad se usan como "barreras" para hacer la partición del espacio.

Para determinar lo que significa alta densidad utilizan un "radio de vecindad".

No se necesita conocer el número de grupos de antemano.

(*) Hoy veremos este tipo de técnicas

Basados en mallas

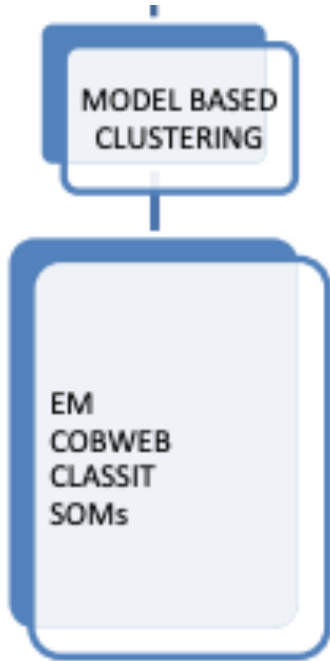


En lugar de agrupar los datos, estos métodos definen una malla que divide el espacio en celdas.

Los datos se colocan en las celdas y si estas celdas satisfacen ciertas condiciones (densidad) se considera que la celda o celdas forman un grupo.

Computacionalmente eficiente.

Basados en modelos



Se asume que los datos están distribuidos en grupos que siguen ciertas distribuciones de probabilidad (por ejemplo, gaussianas), generalmente, un modelo de mezcla.

Los parámetros de los modelos se infieren usando distintas técnicas estadísticas (por ejemplo, máxima verosimilitud o BIC)

Algoritmos basados en densidad

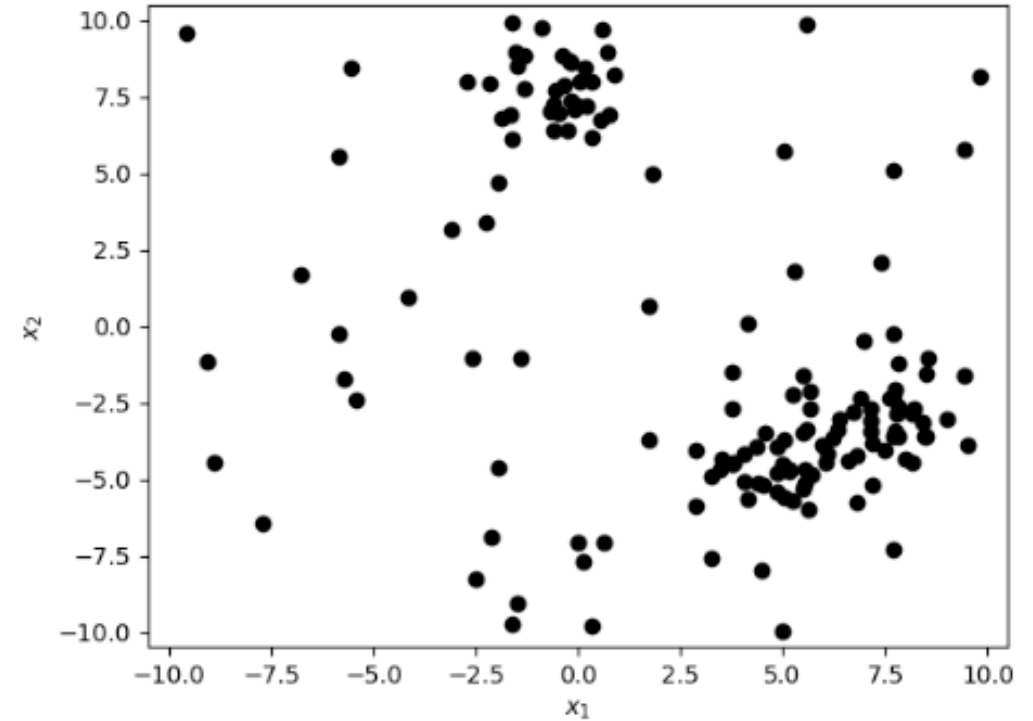
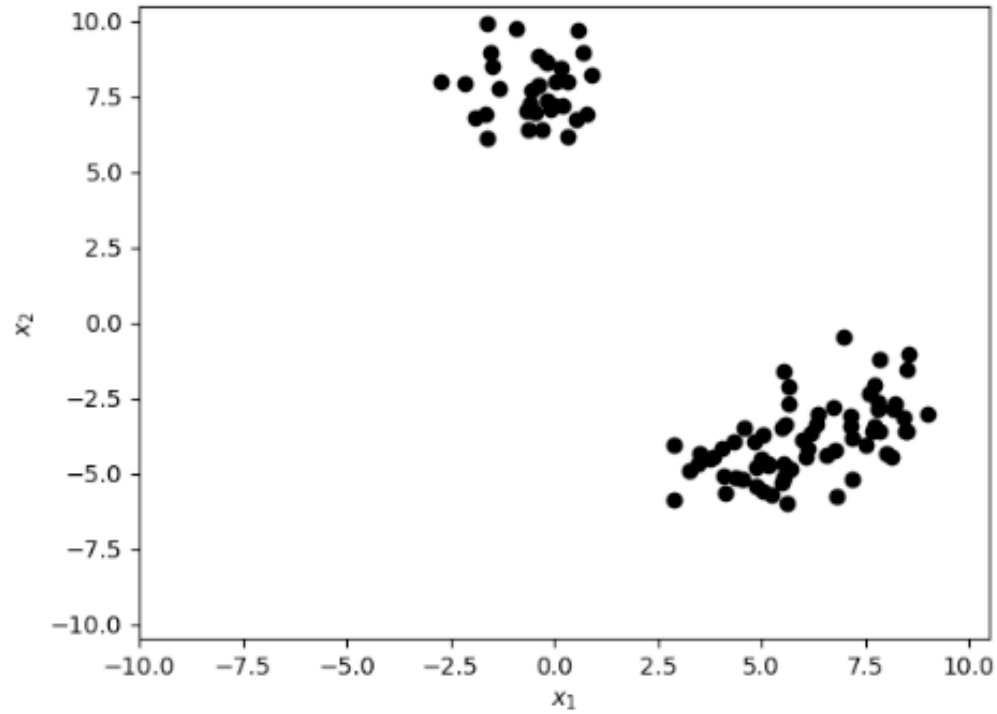
El objetivo es encontrar agrupaciones asumiendo que las densidades de los datos difieren de un grupo a otro y que los grupos definen regiones de alta densidad.

- No requiere conocer el número de grupos previamente.
- Manejan valores atípicos o ruido en los datos.
- No requiere que las estructuras sean esféricas/elípticas

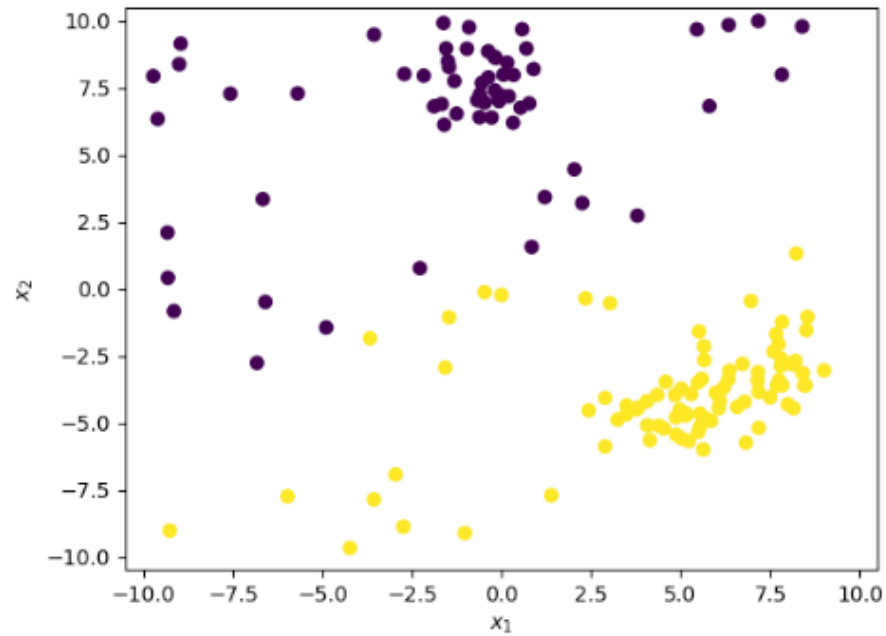
Uno de los algoritmos basados en densidad es DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

¿Por qué agrupamiento basado en densidad?

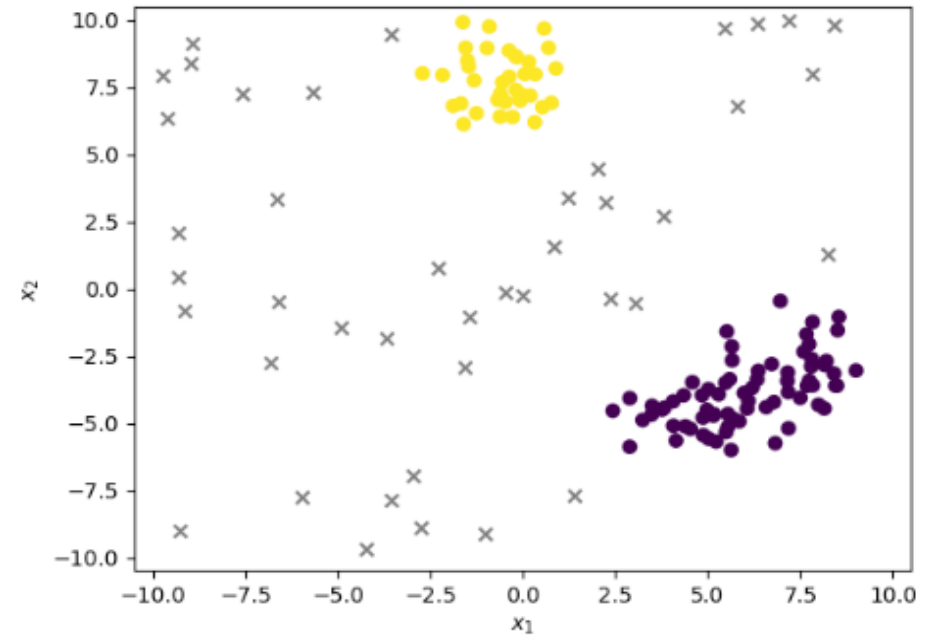
¿Y si los datos tienen ruido?



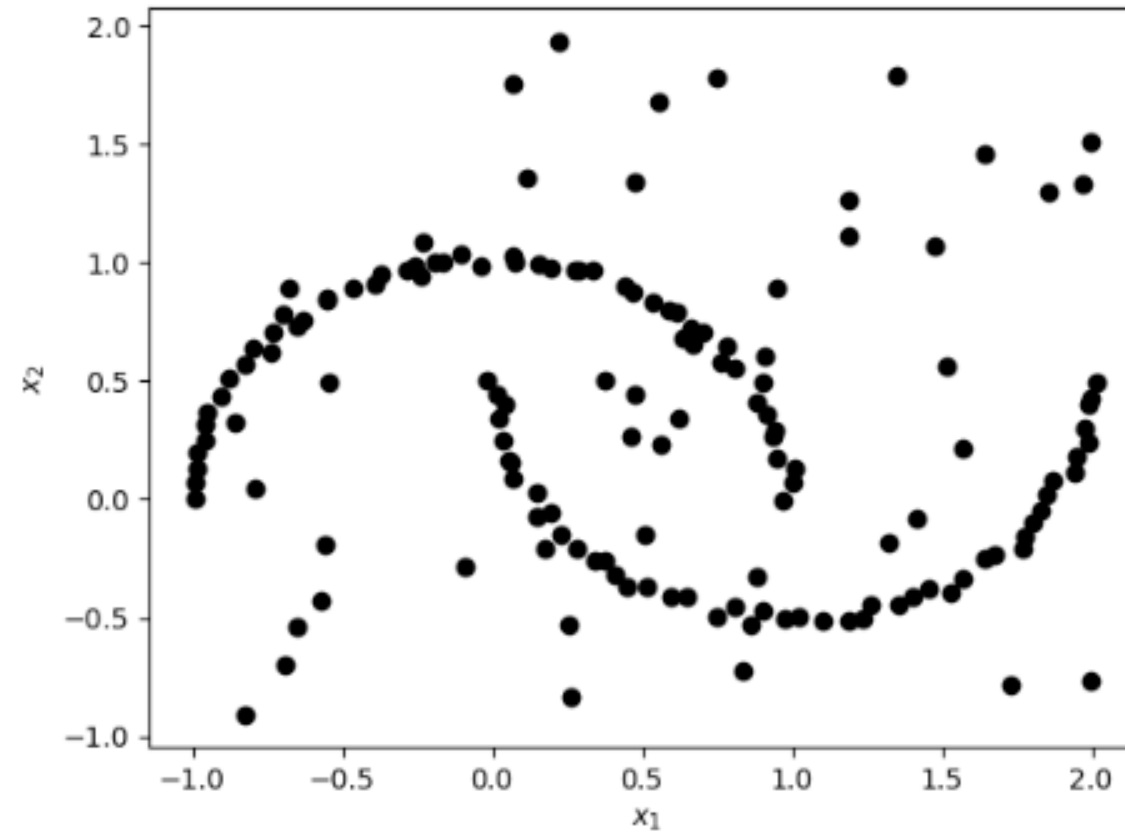
K-Means



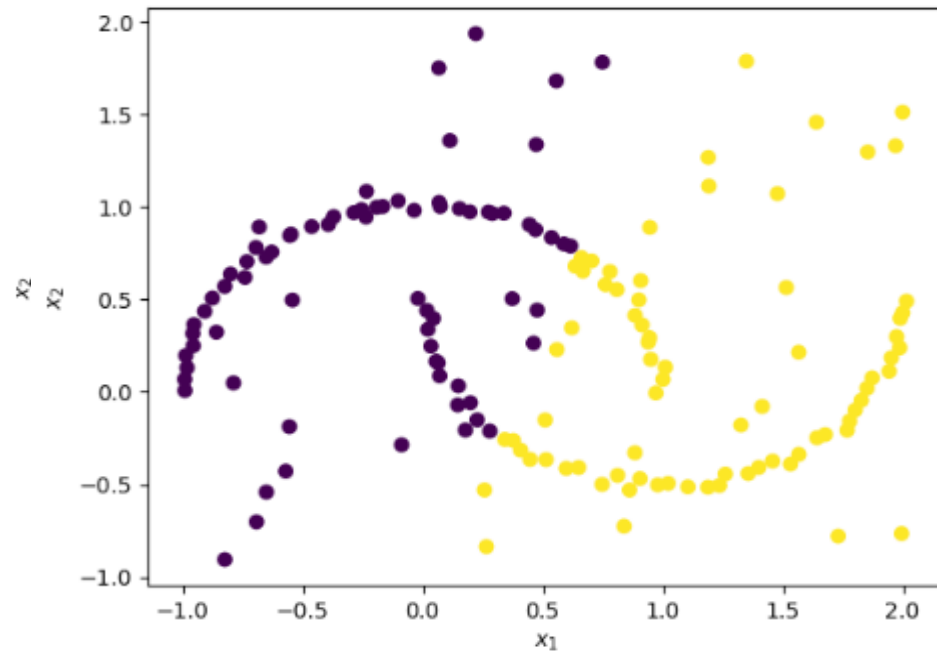
DBSCAN



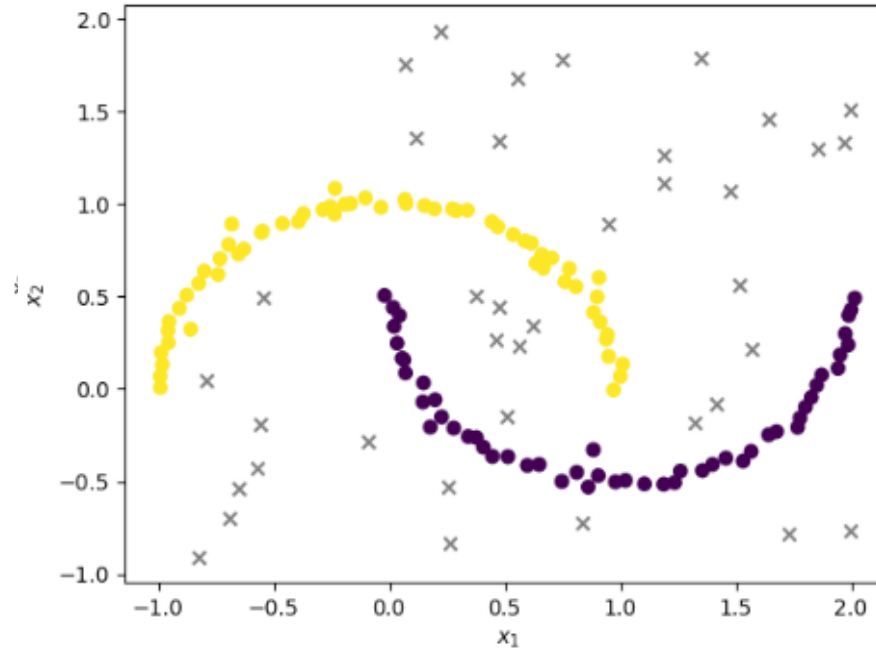
¿Y si, además, los grupos ni siquiera tienen una estructura esférica/elíptica?



K-Means



DBSCAN



DBSCAN

Para comprender el algoritmo DBSCAN hay que definir algunos parámetros fundamentales del método.

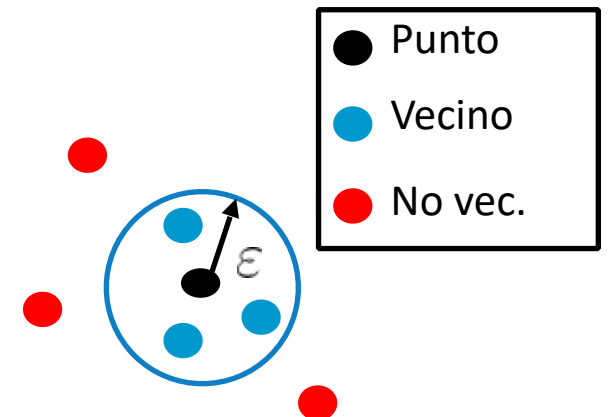
DEFINICIONES

Sean D un conjunto de datos, d la distancia, $\varepsilon \in \mathbb{R}_+$ y $\text{minPts} \in \mathbb{N}$

- **Vecindad de radio ε de un punto**

Dado un punto, la vecindad de radio ε viene dada por aquellos puntos que se encuentran a una distancia igual o inferior a ε

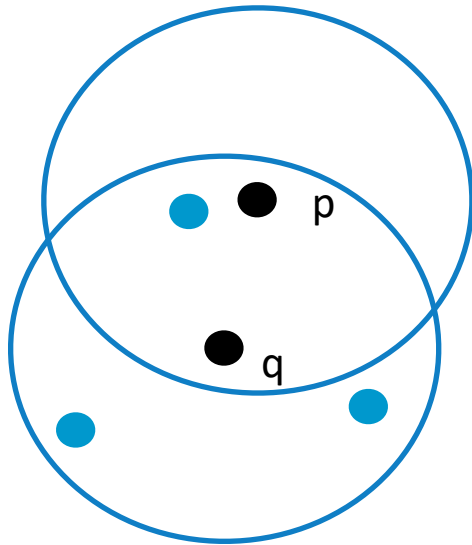
$$N_\varepsilon(p) = \{q \in D : d(p, q) \leq \varepsilon\}$$



- **Puntos densamente alcanzables de forma directa**

Un punto p es densamente alcanzable de forma directa por un punto q si:

- $p \in N_\varepsilon(q)$; p está en la vecindad ε de q
- $|N_\varepsilon(q)| \geq \text{minPts}$; la vecindad de q tiene igual o más puntos que un valor minPts

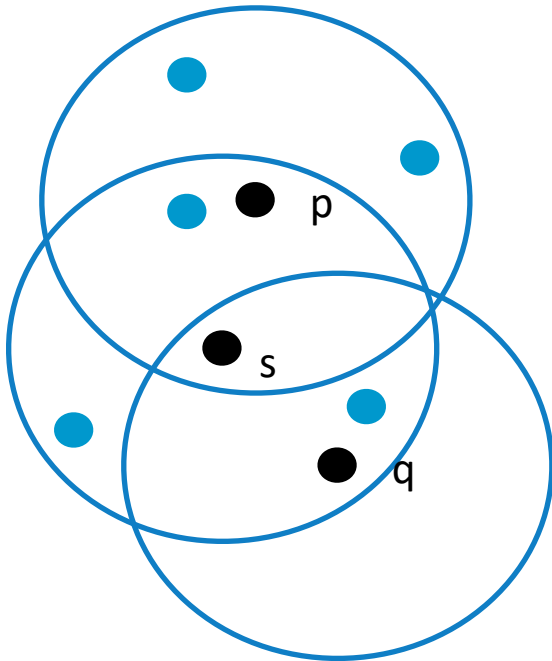


Si $\text{minPts} = 4$ p es densamente alcanzable de forma directa por q pero q no es alcanzable de forma directa por p

- **Punto densamente alcanzable**

Un punto p es densamente alcanzable desde q si existe una sucesión de puntos

$\{q=p_1, p_2, p_3, \dots, p_n=p\}$ tales que p_{i+1} es directamente alcanzable desde p_i .

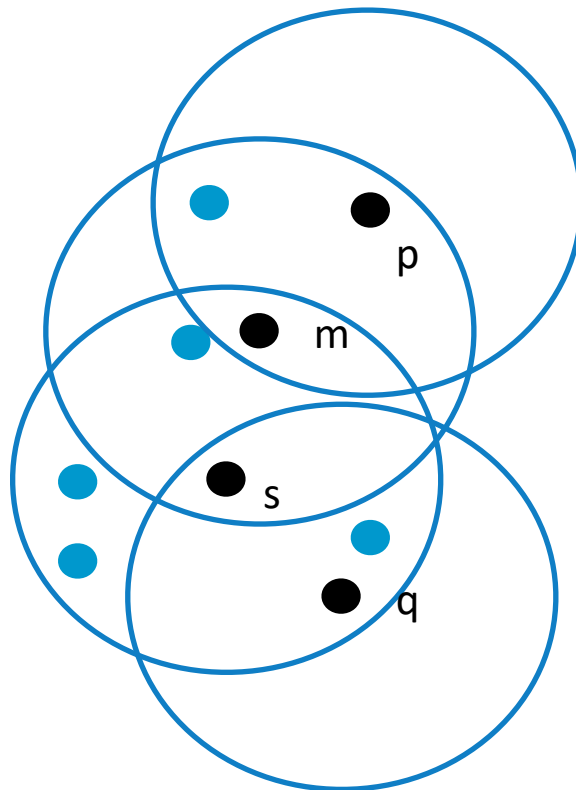


Sea $\text{minPts} = 4$. Entonces:

- $s \in N_\epsilon(p)$ y $|N_\epsilon(p)| \geq \text{minPts} \rightarrow s$ es directamente alcanzable por p .
- $q \in N_\epsilon(s)$ y $|N_\epsilon(q)| \geq \text{minPts} \rightarrow q$ es directamente alcanzable por s .

- **Puntos densamente conectados**

p y q están densamente conectados si existe un punto s tal que tanto p como q son densamente alcanzables desde s .



p y q están densamente conectados

- **Grupo**

Un grupo C es un subconjunto no vacío de los datos que satisface:

- Si p pertenece al grupo C y q es densamente alcanzable desde p , entonces q pertenece al grupo C .
- Si p y q pertenecen al grupo C , p está densamente conectado con q .

- **Ruido**

Todos los puntos del conjunto de datos que no forman parte de ningún grupo.

Con este procedimiento, DBSCAN clasifica los puntos pertenecientes a un grupo como:

- **Puntos centrales**

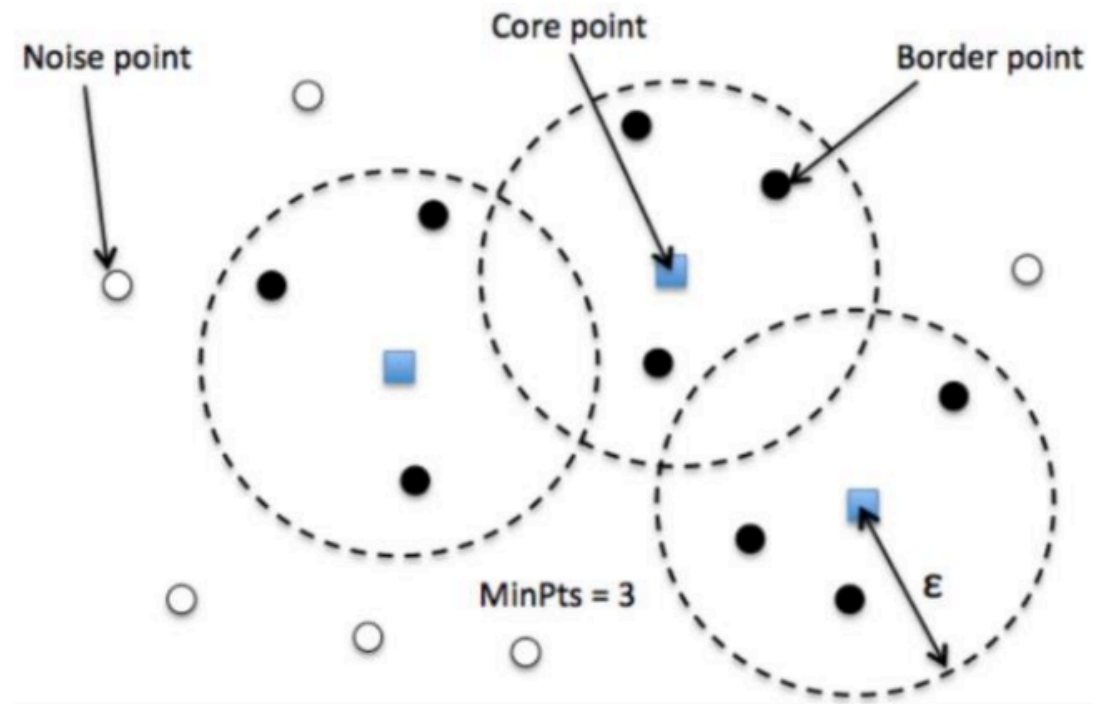
Puntos que tienen un número mayor o igual a minPts en su vecindario ϵ

- **Puntos fronterizos**

Puntos que no son centrales pero están dentro del vecindario de, al menos, un punto central

- **Ruido**

Puntos que no son centrales ni fronterizos



<https://www.kdnuggets.com/2020/04/dbscan-clustering-algorithm-machine-learning.html>

De las definiciones anteriores se deducen los siguientes lemas:

1. Sea $p \in D$ tal que $|N_\varepsilon(p)| \geq \text{minPts}$. Entonces, el conjunto de puntos densamente alcanzables desde p forman un grupo.
2. Sea $C \subset D$ y sea p un punto central, $p \in C$ tal que $|N_\varepsilon(p)| \geq \text{minPts}$, entonces, todos los puntos densamente alcanzables por p pertenecen a C

PROCEDIMIENTO

Dados minPts y ε

$C = 0$

Para cada p no visitado

 Marcar p como visitado

 Construir $N_\varepsilon(p)$ (vecindad)

 Si $|N_\varepsilon(p)| < \text{minPts} \rightarrow$ marcar p como ruido

 Si no

$C = C + 1$

 ExtenderGrupo($p, N_\varepsilon(p), C$)

ExtenderGrupo ($p, N_\varepsilon(p), C$)

 Añadir p al grupo C

 Para cada p' de la vecindad

 Si p' no se ha visitado

 Marcar p' como visitado

 Construir $N_\varepsilon(p')$

 Si $|N_\varepsilon(p')| \geq \text{minPts}$

$N_\varepsilon(p) \leftarrow N_\varepsilon(p) \cup N_\varepsilon(p')$

 Si p' no pertenece a ningún grupo

$C \leftarrow C \cup p'$

Ejemplo:

Seleccionamos $\varepsilon = 1.5$; $\text{minPts} = 2$

Sean los puntos de datos

$$A = (1, 1)$$

$$B = (2, 1)$$

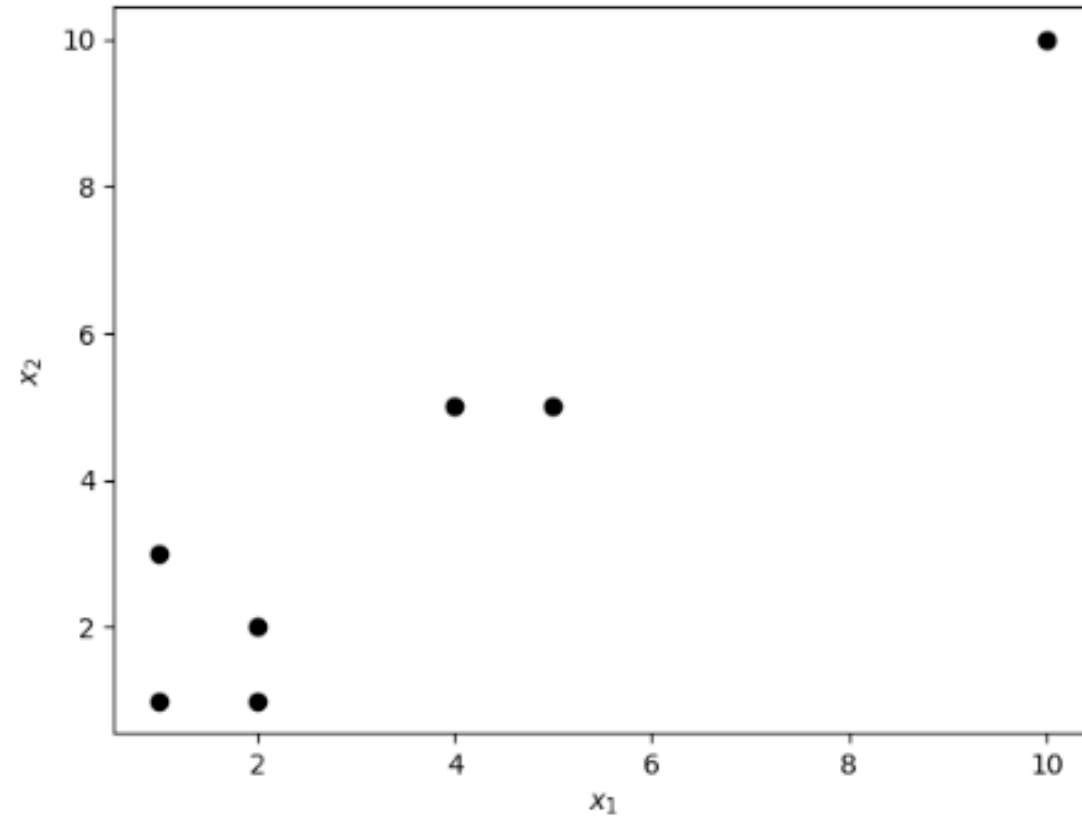
$$C = (1, 3)$$

$$D = (2, 2)$$

$$E = (5, 5)$$

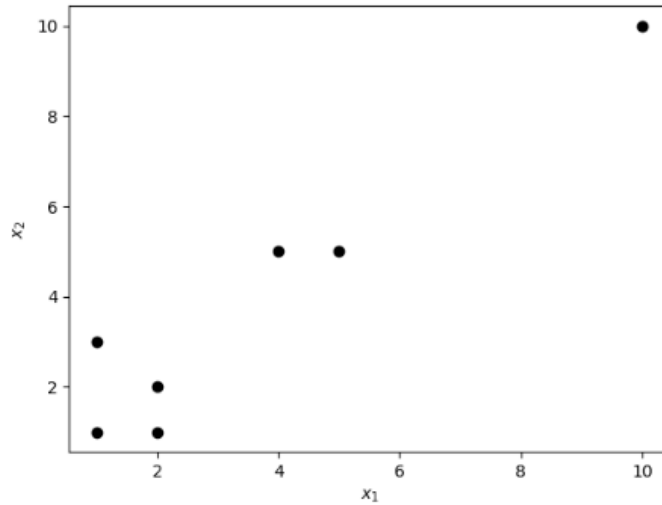
$$F = (4, 5)$$

$$G = (10, 10)$$



Calculamos la matriz de distancias

$A = (1, 1)$
 $B = (2, 1)$
 $C = (1, 3)$
 $D = (2, 2)$
 $E = (5, 5)$
 $F = (4, 5)$
 $G = (10, 10)$

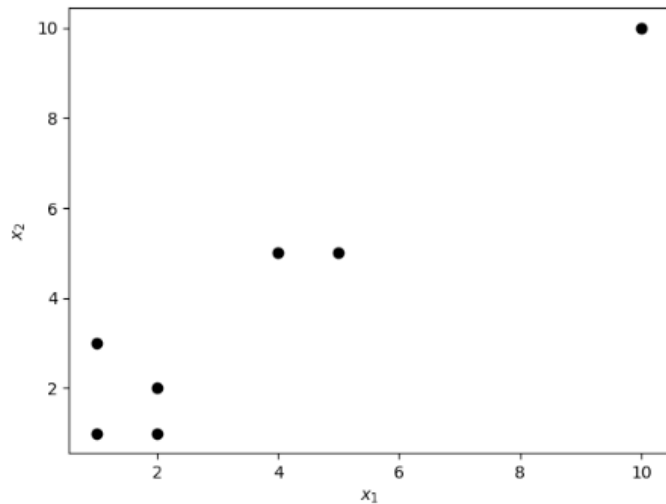


$$M_d = \begin{pmatrix} 0 & 1 & 2 & 1.41 & 5.66 & 5 & 12.73 \\ 1 & 0 & 2.24 & 1 & 5 & 4.47 & 12.04 \\ 2 & 2.24 & 0 & 1.41 & 4.47 & 3.61 & 11.4 \\ 1.41 & 1 & 1.41 & 0 & 4.24 & 3.61 & 11.31 \\ 5.66 & 5 & 4.47 & 4.24 & 0 & 1 & 7.07 \\ 5 & 4.47 & 3.61 & 3.61 & 1 & 0 & 7.81 \\ 12.73 & 12.04 & 11.4 & 11.31 & 7.07 & 7.81 & 0 \end{pmatrix}$$

$A = (1, 1)$
 $B = (2, 1)$
 $C = (1, 3)$
 $D = (2, 2)$
 $E = (5, 5)$
 $F = (4, 5)$
 $G = (10, 10)$

$\varepsilon = 1.5; \text{minPts} = 2$

$$M_d = \begin{pmatrix} 0 & 1 & 2 & 1.41 & 5.66 & 5 & 12.73 \\ 1 & 0 & 2.24 & 1 & 5 & 4.47 & 12.04 \\ 2 & 2.24 & 0 & 1.41 & 4.47 & 3.61 & 11.4 \\ 1.41 & 1 & 1.41 & 0 & 4.24 & 3.61 & 11.31 \\ 5.66 & 5 & 4.47 & 4.24 & 0 & 1 & 7.07 \\ 5 & 4.47 & 3.61 & 3.61 & 1 & 0 & 7.81 \\ 12.73 & 12.04 & 11.4 & 11.31 & 7.07 & 7.81 & 0 \end{pmatrix}$$



Iteración 1

$V = \{A\}$ (visitados)

$N_\varepsilon(A) = \{B, D\} \rightarrow |N_\varepsilon(A)| = 2 \geq \text{minPts} \Rightarrow \text{Extender}(A, N_\varepsilon(A), G_1)$

Extensión:

$G_1 = \{A\}$

- Empezamos visitando $B \rightarrow V = \{A, B\}$

$N_\varepsilon(B) = \{A, D\} \rightarrow |N_\varepsilon(B)| = 2 \geq \text{minPts} \Rightarrow N_\varepsilon(A) = N_\varepsilon(A) \cup N_\varepsilon(B) = \{A, B, D\}$

Como B no pertenece a ningún grupo, entonces $G_1 = \{A, B\}$

- Visitamos $D \rightarrow V = \{A, B, D\}$

$N_\varepsilon(D) = \{A, B, C\} \rightarrow |N_\varepsilon(D)| = 3 \geq \text{minPts} \Rightarrow$

$N_\varepsilon(A) = N_\varepsilon(A) \cup N_\varepsilon(D) = \{A, B, C, D\}$. Como D no pertenece a ningún grupo, entonces $G_1 = \{A, B, D\}$.

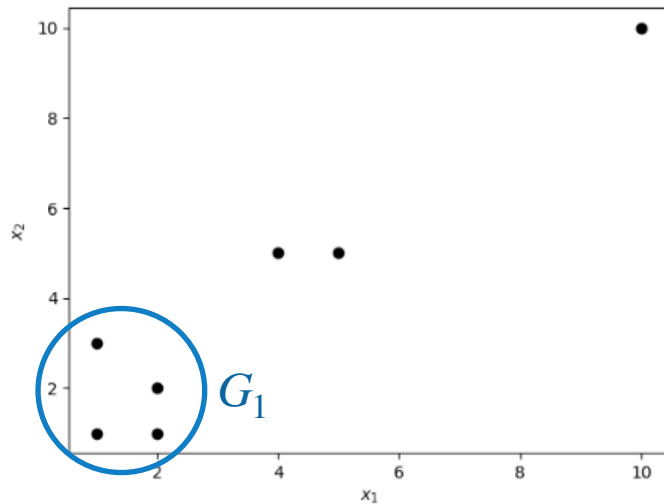
- Visitamos $C \rightarrow V = \{A, B, C, D\}$.

$N_\varepsilon(C) = \{D\} \rightarrow |N_\varepsilon(C)| = 1 < \text{minPts}$. Como C no pertenece a ningún grupo, entonces $G_1 = \{A, B, C, D\}$

$A = (1, 1)$
 $B = (2, 1)$
 $C = (1, 3)$
 $D = (2, 2)$
 $E = (5, 5)$
 $F = (4, 5)$
 $G = (10, 10)$

$\varepsilon = 1.5$; minPts = 2

$$M_d = \begin{pmatrix} 0 & 1 & 2 & 1.41 & 5.66 & 5 & 12.73 \\ 1 & 0 & 2.24 & 1 & 5 & 4.47 & 12.04 \\ 2 & 2.24 & 0 & 1.41 & 4.47 & 3.61 & 11.4 \\ 1.41 & 1 & 1.41 & 0 & 4.24 & 3.61 & 11.31 \\ 5.66 & 5 & 4.47 & 4.24 & 0 & 1 & 7.07 \\ 5 & 4.47 & 3.61 & 3.61 & 1 & 0 & 7.81 \\ 12.73 & 12.04 & 11.4 & 11.31 & 7.07 & 7.81 & 0 \end{pmatrix}$$



Iteración 2

$V = \{A, B, C, D, E\}$ (vistamos un nuevo punto no visitado, E)

$N_\varepsilon(E) = \{F\} \rightarrow |N_\varepsilon(E)| = 1 < \text{minPts} \Rightarrow \text{Ruido}$

Iteración 3

$V = \{A, B, C, D, E, F\}$ (vistamos un nuevo punto no visitado, F)

$N_\varepsilon(F) = \{E\} \rightarrow |N_\varepsilon(F)| = 1 < \text{minPts} \Rightarrow \text{Ruido}$

Iteración 4

$V = \{A, B, C, D, E, F, G\}$ (vistamos un nuevo punto no visitado, G)

$N_\varepsilon(G) = \emptyset \rightarrow |N_\varepsilon(G)| = 0 < \text{minPts} \Rightarrow \text{Ruido}$

Clasificación final

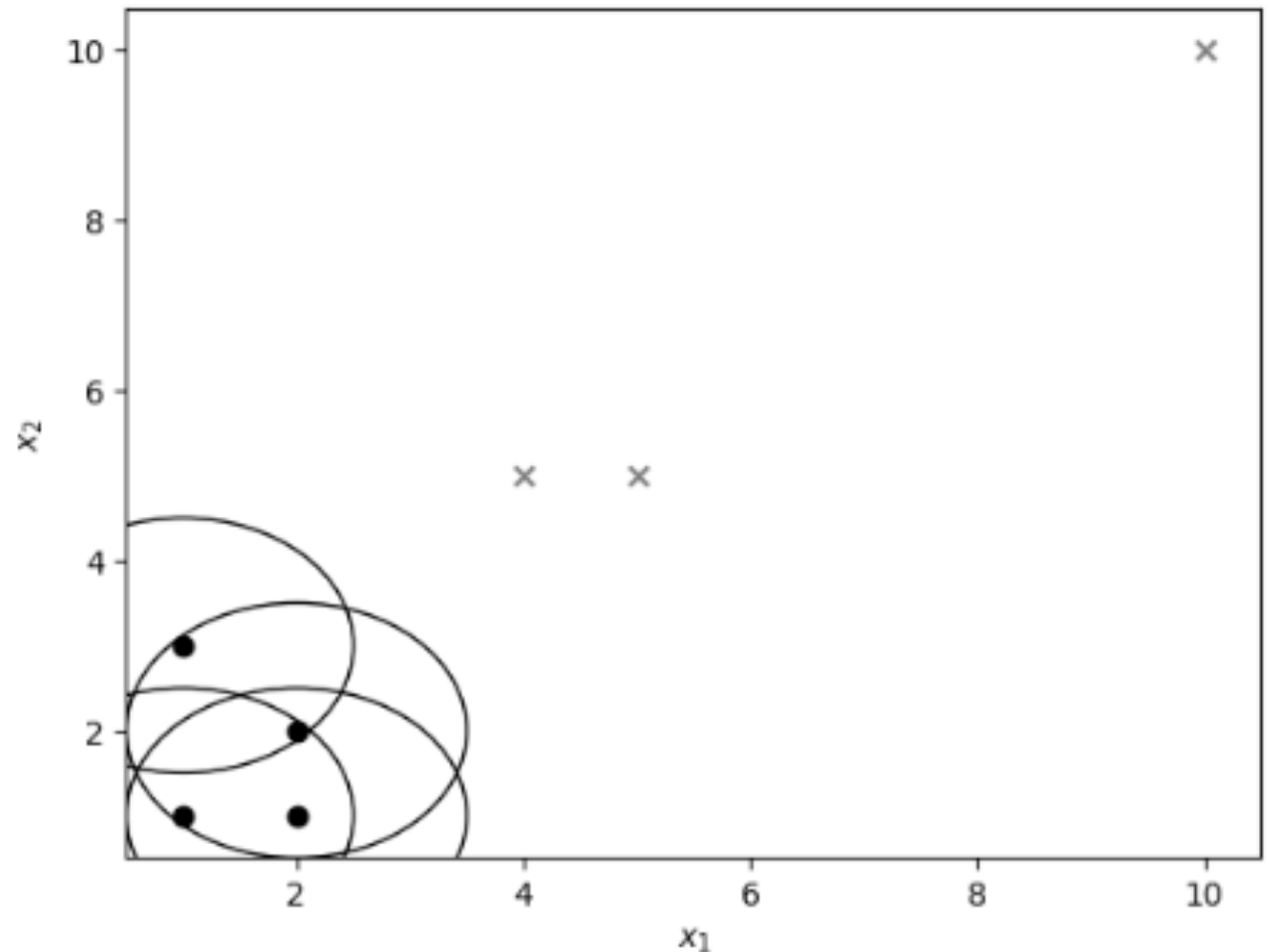
Se tiene un grupo $G_1 = \{A, B, C, D\}$

Como $\min Pts \leq N_\epsilon(A), N_\epsilon(B), N_\epsilon(D)$, los puntos A, B y D son puntos centrales.

Como C pertenece a un grupo pero no es central, es fronterizo.

Los puntos E, F y G son ruido

$$\begin{aligned} A &= (1, 1) \\ B &= (2, 1) \\ C &= (1, 3) \\ D &= (2, 2) \\ E &= (5, 5) \\ F &= (4, 5) \\ G &= (10, 10) \end{aligned}$$



Aviso: en scikit-learn, el parámetro no es *minPts* sino *min_samples*, esto incluyen los puntos centrales, por tanto, corresponde a *minPts+1* en los cálculos realizados aquí.



www.unir.net

Hiperparámetros en DBSCAN

Radio de vecindad ϵ

Distancia máxima entre dos puntos para considerarse vecinos.

- Si es demasiado grande:
 - Muchos grupos se unirán en uno más grande, perdiendo información.
 - Regiones de baja densidad pueden unirse en un solo grupo.
 - Puntos que son ruido pueden fusionarse con grupos.
- Si es demasiado pequeño:
 - Muchos puntos serán asignados como ruido
 - Sólo los grupos más densos se considerarán grupos como tales, perdiendo estructuras que sí forman grupos.

Número mínimo de puntos minPts

Número mínimo de puntos que debe tener un punto en una distancia ε para considerarse un punto central (i.e, el número mínimo de puntos -1 que debe tener un grupo)

- Si es muy grande:
 - Detección de los grupos más densos.
 - Muchos grupos no se considerarán así ya que se exigen más para formar grupo, estos puntos se considerarán ruido.
- Si es muy pequeño:
 - Regiones poco densas pueden considerarse grupos.
 - Se pueden formar grupos de puntos que son ruido.
 - Sobresegmentación (dependiendo del radio) de los grupos.

Elección de los hiperparámetros

- **minPts**

Existe una regla de uso (rule of thumb): dado un conjunto de datos donde los datos son de dimensión D , entonces

$$\text{minPts} \geq D + 1$$

En general, una elección de $\text{minPts} = 2D$ es un buen punto de partida.

Idealmente, nos lo daría el conocimiento del problema que estamos tratando.

En ausencia de este conocimiento, se puede partir de las reglas mencionadas y probar variaciones observando valores de distintas métricas.

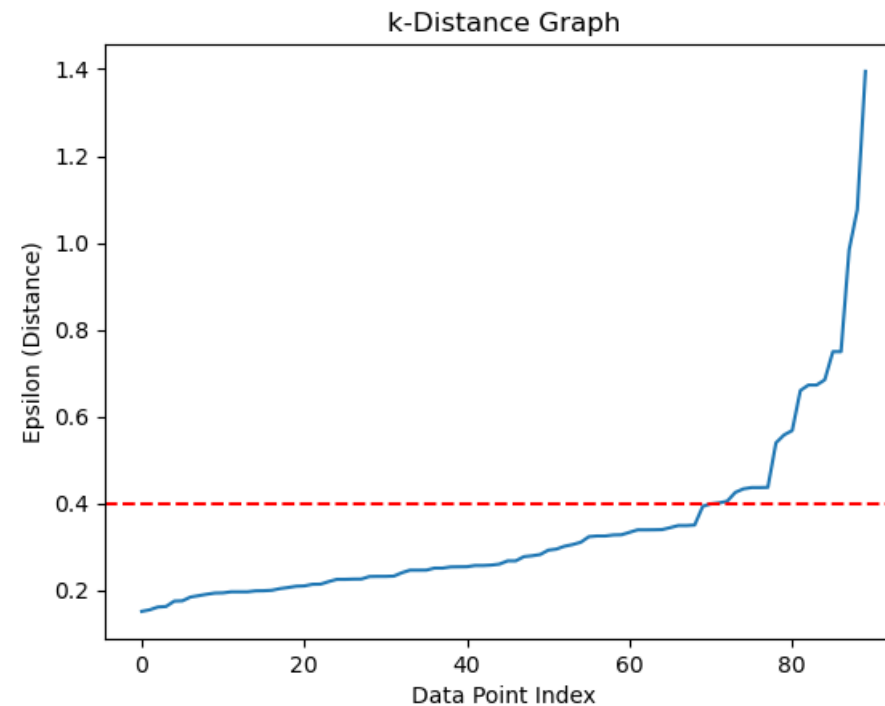
- **Radio ε**

Al igual que con minPts, lo da los conocimientos previos del problema y/o distintas pruebas.

Una buena aproximación para determinar el radio dado el número mínimo de puntos es un gráfico de las distancias a los k-vecinos más próximos. La idea detrás de este gráfico es que:

- Los puntos interiores de los grupos tienen los k-vecinos más o menos a la misma distancia.
- El ruido tiene los k-vecinos más alejados.

Con este método, se usa $k = \text{minPts}$.



Evaluación de las agrupaciones

Para medir la bondad del agrupamiento se suelen usar las siguientes métricas (recordemos que aquí en el aprendizaje no supervisado no se disponen de las métricas reales).

- **Índice de silueta** (introducido en el Tema 2) No es fiable cuando hay ruido.

$$a_i = \frac{1}{|C_i| - 1} \sum_{j \in \{C_j = C_i\}} d(x_i, x_j)$$

Distancia media del punto a los puntos que pertenecen al mismo grupo

$$b_i = \min \left[\frac{1}{|C_k|} \sum_{k \in \{C_k \neq C_i\}} d(x_i, x_j) \right]$$

Mínimo de las distancias medias del punto a los puntos de los otros grupos.

$$s_i = \frac{b_i - a_i}{\max[a_i, b_i]}$$

$$s(k) = \frac{1}{N} \sum_i s_i$$

- **Índice de Calinski-Harabasz**

No depende de las etiquetas reales (métrica interna). También se conoce como el criterio de la razón de varianzas.

Sean

$D = \{\vec{x}_i\}_{i=1}^N$	conjunto de datos
k	número de grupos
$\vec{c}_i$	centroide el grupo i-ésimo
n_i	Número de datos pertenecientes al grupo i-ésimo
d	distancia
$\vec{c}$	centroide de todos los datos

Cuanto mayor es CH, mejor es la agrupación realizada.

$$CH = \frac{\frac{1}{k-1} \sum_{i=1}^k d^2(\vec{c}_i, \vec{c})}{\frac{1}{n-k} \sum_{i=1}^k \sum_{j=1}^{n_i} d^2(\vec{x}_j, \vec{c}_i)}$$

- **Índice de Davies-Bouldin**

Es otra métrica interna

El índice de Davies-Bouldin se define como (siendo n_c el número de grupos formados):

$$DB = \frac{1}{n_c} \sum_{i=1}^{n_c} \max_{j \neq i} \{R_{ij}\}$$

donde

$$R_{ij} = \frac{S_i + S_j}{d_{ij}}; \quad S_i = \frac{1}{n_i} \sum_{k=1}^{n_i} d(\vec{x}_k, \vec{c}_i); \quad d_{ij} = d(\vec{c}_i, \vec{c}_j)$$

Distancia media de los elementos del grupo i-ésimo a su
centroide

Distancia entre los centroides de los grupos i-ésimo y j-
ésimo

Por tanto, el índice de Davies-Bouldin es el promedio de los R_{ij} donde para cada grupo se elige para calcular R_{ij} al grupo que maximiza R_{ij} , esto es, el peor caso.

NOTA: DB es mejor cuanto menor es.

- **Índice Rand**

A diferencia de las anteriores, esta métrica no es interna. Sin embargo, puede usarse como métrica para comparar dos agrupaciones distintas.

Sea una agrupación dada por una técnica y otra de referencia (en principio, la agrupación real, sin embargo, se puede usar la dada por otra técnica o la misma con otros parámetros).

Se define el índice Rand como:

$$R = \frac{a + b}{a + b + c + d} = \frac{a + b}{\binom{N}{2}}$$

donde

N	número total de datos
a	Número de pares que están en el mismo grupo en ambas agrupaciones
b	Número de pares en distintos grupos en ambas agrupaciones
c	Número de pares juntos en la primera técnica y en distinto en el de referencia
d	Número de pares en grupos distintos por la primera técnica pero juntos en el de referencia

$$R \in [0,1] \rightarrow R = \begin{cases} 1 & \text{si coinciden perfectamente} \\ 0 & \text{si no hay ningún acuerdo} \end{cases}$$

Para tener en cuenta coincidencias aleatorias, se define el índice Rand ajustado como:

$$ARI = \frac{R - E[R]}{\max\{R\} - E[R]} = \frac{2(ab - cd)}{(a + c)(b + c) + (a + d)(b + d)}$$

$$ARI = \begin{cases} 1 & \text{si coinciden perfectamente} \\ 0 & \text{si las coincidencias pueden atribuirse al azar} \\ < 0 & \text{peor que el azar} \end{cases}$$

Ejemplo: se han realizado dos agrupamientos empleando dos técnicas distintas sobre un conjunto de 7 puntos.

Método 1

Punto	Grupo
A	1
B	1
C	1
D	2
E	2
F	3
G	3

Método 2

Punto	Grupo
A	2
B	2
C	2
D	1
E	1
F	1
G	3

$$R = \frac{a + b}{a + b + c + d} = \frac{a + b}{\binom{N}{2}}$$

Pares que ambos métodos ponen en el mismo grupo:

AB, AC, BC y DE $\Rightarrow a = 4$

Pares que ambos métodos asignan a distintos grupos:

AD, AE, AF, AG, BD, BE, BF, BG, CD, CE, CF, CG,
DG y EG $\Rightarrow b = 14$

$$R = \frac{4 + 14}{\binom{7}{2}} = \frac{18}{\frac{7!}{2!5!}} = \frac{18}{21} = 0.86$$

Método 1

Punto	Grupo
A	1
B	1
C	1
D	2
E	2
F	3
G	3

Método 2

Punto	Grupo
A	2
B	2
C	2
D	1
E	1
F	1
G	3

$$ARI = \frac{R - E[R]}{\max\{R\} - E[R]} = \frac{2(ab - cd)}{(a + c)(b + c) + (a + d)(b + d)}$$

De antes ya conocemos $a = 4$ y $b = 14$.

Pares unidos con el método 1 y separados con el método 2:

$$FG \Rightarrow c = 1$$

Pares separados con el método 1 y unidos con el método 2:

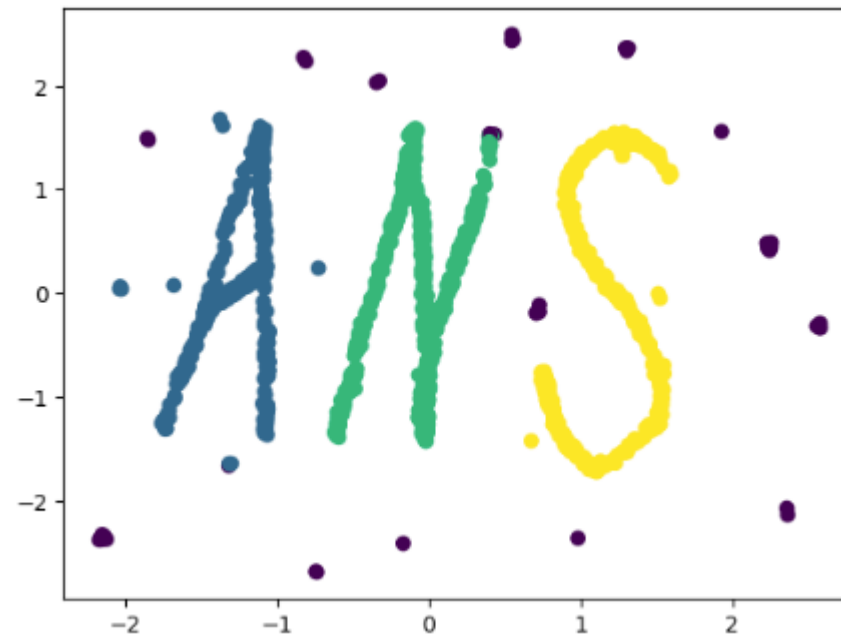
$$DF \text{ y } EF \Rightarrow d = 2$$

$$ARI = \frac{2(4 \cdot 14 - 1 \cdot 2)}{(4 + 1)(14 + 1) + (4 + 2)(14 + 2)} = \frac{108}{171} = 0.63$$

Ventajas/desventajas de DBSCAN

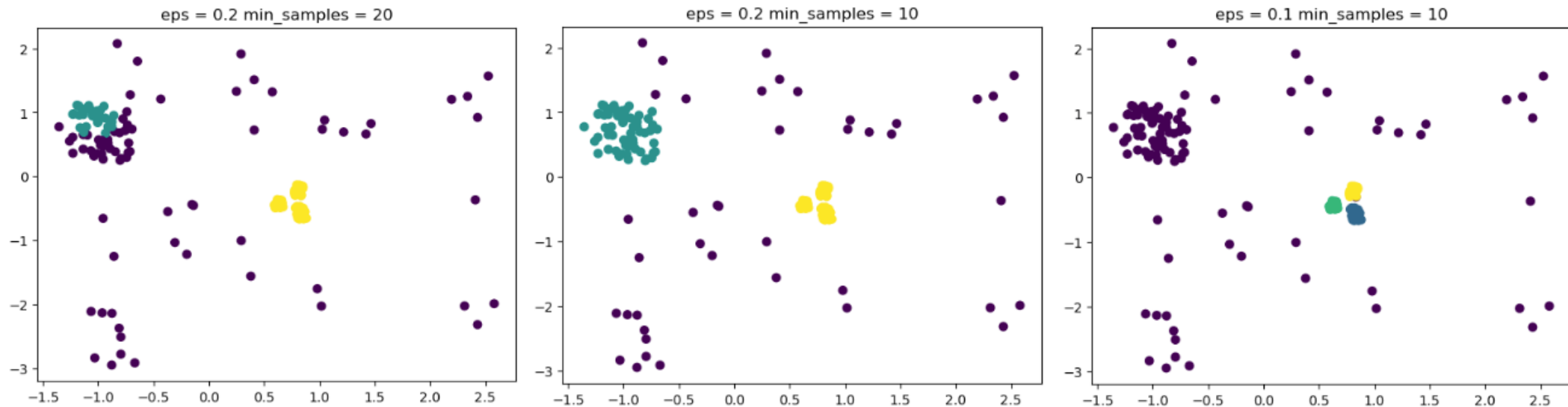
VENTAJAS

- No es necesario especificar el número de grupos
- Puede trabajar con grupos de distintos tamaños y formas
- Puede manejar valores atípicos/ruido.



PROBLEMAS

- Sensible a los hiperparámetros.
- No trabaja bien cuando en el conjunto de datos hay grupos con densidades muy distintas.
- La asignación puede depender de cómo se recorran los puntos para algunas implementaciones. Si un punto ya ha sido asignado a un grupo, no puede reasignarse.



Variaciones de DBSCAN

KDTree

- Un KDTree (k dimensional tree) es una estructura que se utiliza para mejorar la búsqueda en un conjunto de datos.
- Tiene una estructura de árbol que se construye mediante particiones binarias.

Construcción: sea un conjunto de datos de k-dimensiones

- Se elige una de las dimensiones.
- Se calcula la mediana de los datos para esa dimensión y se coloca en el nodo el dato mediana.
- Se dividen los datos en aquellos con un valor menor (izquierda) y un valor mayor (o igual) a la derecha.
- Se continua con la siguiente dimensión

Ejemplo: construcción del KDTree para los siguientes datos

$P_1(1, 3)$

$P_2(1, 8)$

$P_3(2, 2)$

$P_4(2, 10)$

$P_5(3, 6)$

$P_6(4, 1)$

$P_7(5, 4)$

$P_8(6, 8)$

$P_9(7, 4)$

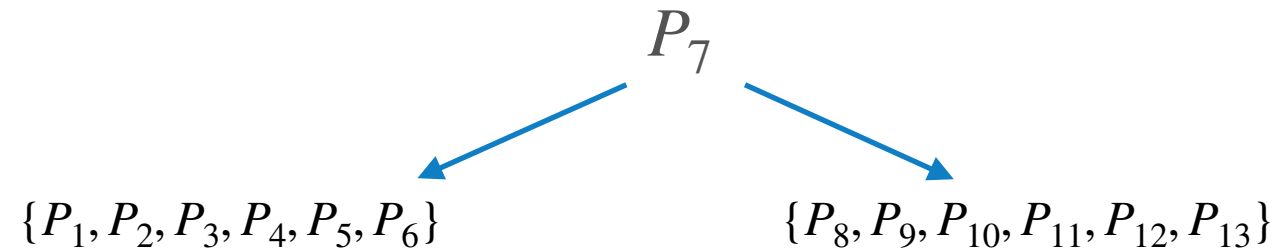
$P_{10}(7, 7)$

$P_{11}(8, 2)$

$P_{12}(8, 5)$

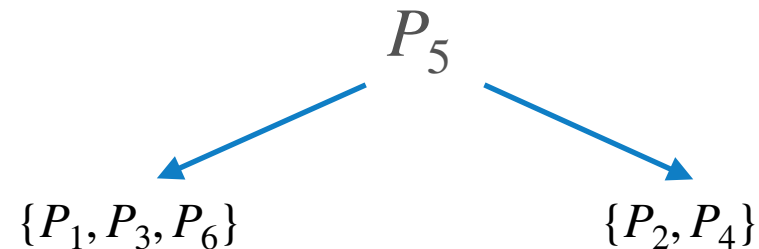
$P_{13}(9, 9)$

Valores en **x**: $\{1, 1, 2, 2, 3, 4, 5, 6, 7, 7, 8, 8, 9\} \rightarrow \text{mediana} = 5 \rightarrow P_7(5, 4)$

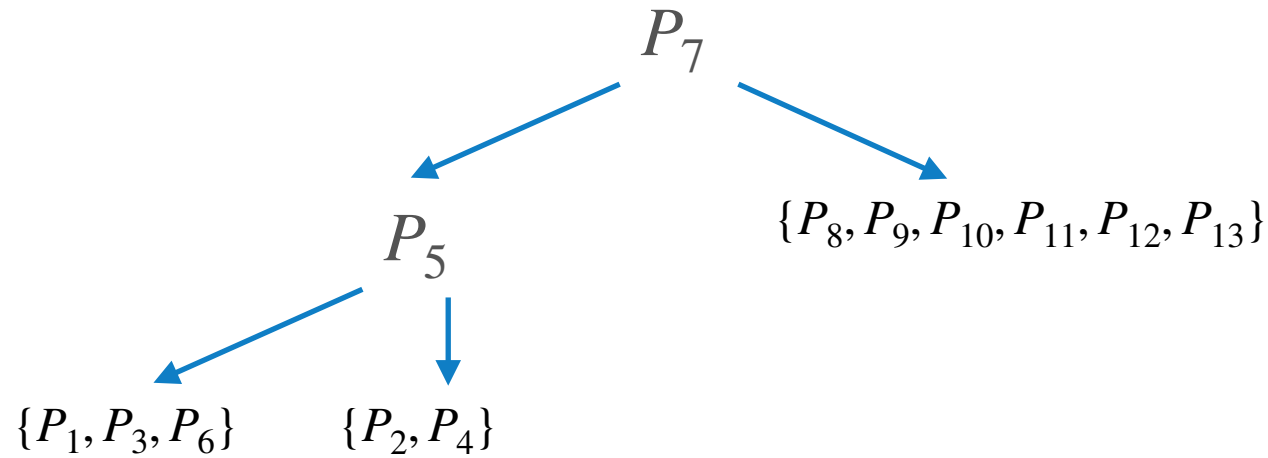


Continuamos con la rama izquierda.

Valores en **y** (izquierda): $\{1, 2, 3, 6, 8, 10\} \rightarrow \text{mediana} = 6 \rightarrow P_5(3, 6)$

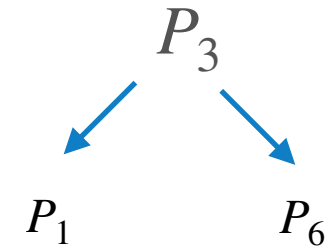


$P_1(1, 3)$
 $P_2(1, 8)$
 $P_3(2, 2)$
 $P_4(2, 10)$
 $P_5(3, 6)$
 $P_6(4, 1)$
 $P_7(5, 4)$
 $P_8(6, 8)$
 $P_9(7, 4)$
 $P_{10}(7, 7)$
 $P_{11}(8, 2)$
 $P_{12}(8, 5)$
 $P_{13}(9, 9)$



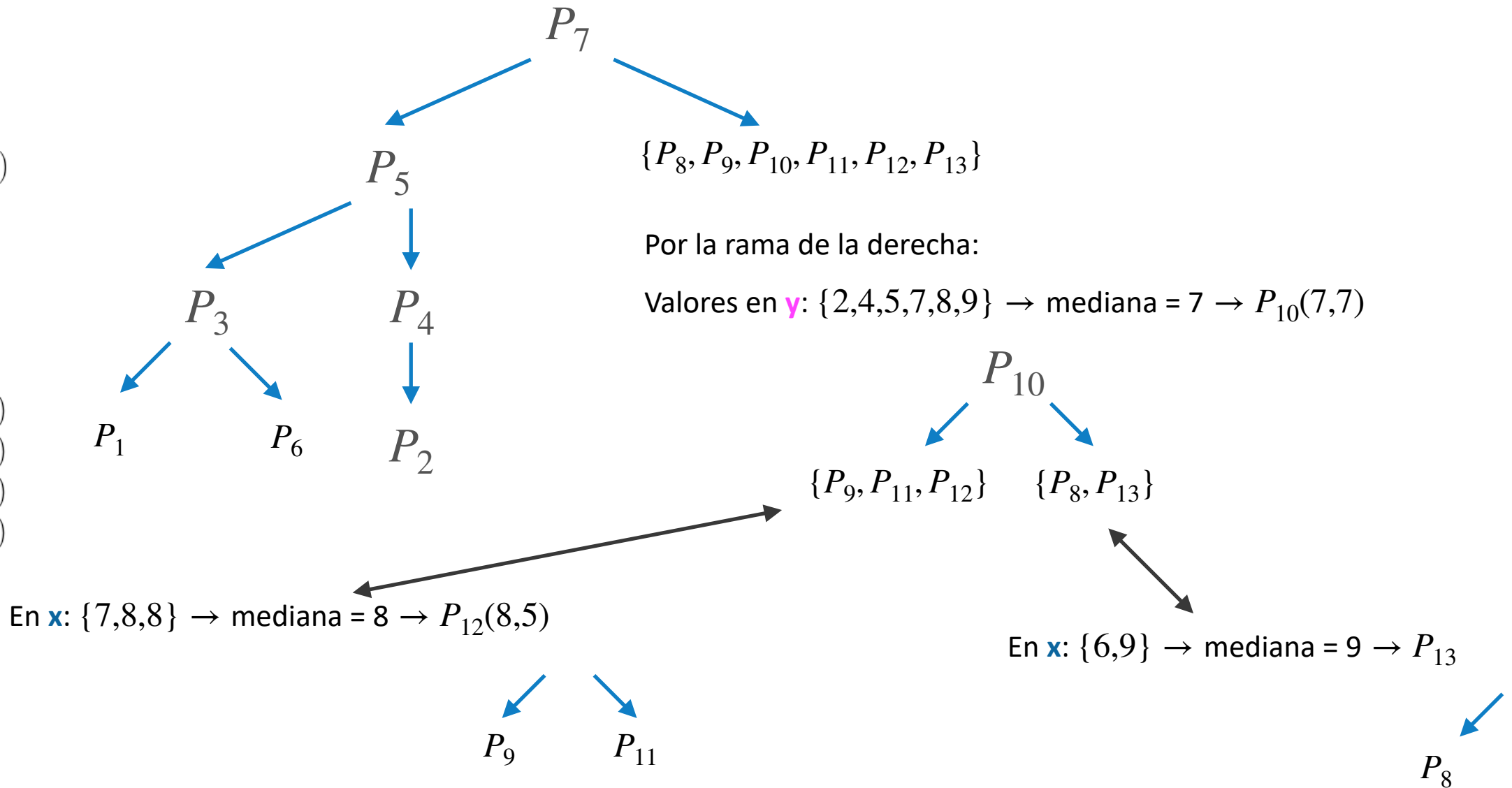
Continuamos con la rama izquierda.

Valores en **x** (izquierda): $\{1, 2, 4\} \rightarrow \text{mediana} = 2 \rightarrow P_3(2, 2)$

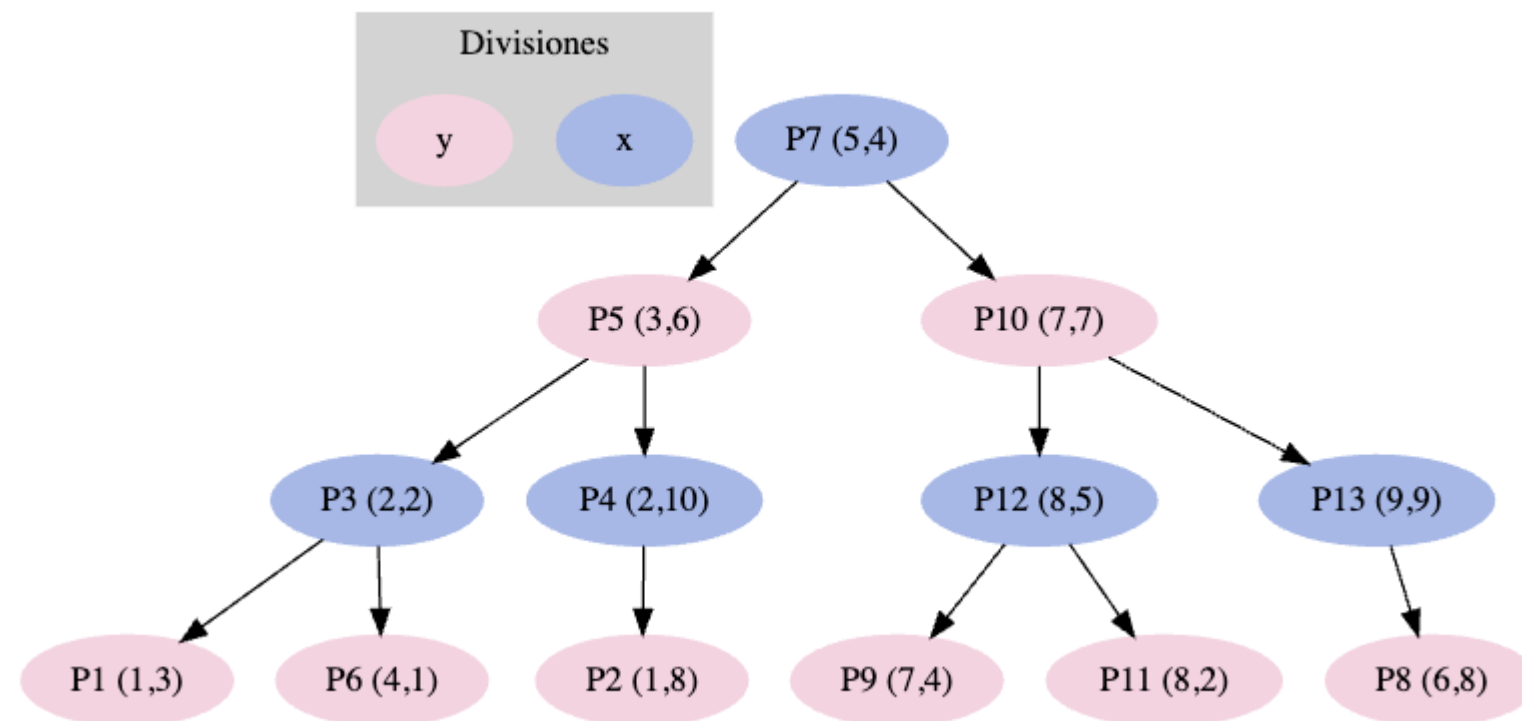


En la rama de la derecha de P_5 , ordenando en **x** se tiene $P_4 \rightarrow P_2$

$P_1(1, 3)$
 $P_2(1, 8)$
 $P_3(2, 2)$
 $P_4(2, 10)$
 $P_5(3, 6)$
 $P_6(4, 1)$
 $P_7(5, 4)$
 $P_8(6, 8)$
 $P_9(7, 4)$
 $P_{10}(7, 7)$
 $P_{11}(8, 2)$
 $P_{12}(8, 5)$
 $P_{13}(9, 9)$



$P_1(1, 3)$
 $P_2(1, 8)$
 $P_3(2, 2)$
 $P_4(2, 10)$
 $P_5(3, 6)$
 $P_6(4, 1)$
 $P_7(5, 4)$
 $P_8(6, 8)$
 $P_9(7, 4)$
 $P_{10}(7, 7)$
 $P_{11}(8, 2)$
 $P_{12}(8, 5)$
 $P_{13}(9, 9)$



Gráficamente

$$P_1(1, 3)$$

$$P_2(1, 8)$$

$$P_3(2, 2)$$

$$P_4(2, 10)$$

$$P_5(3, 6)$$

$$P_6(4, 1)$$

$$P_7(5, 4)$$

$$P_8(6, 8)$$

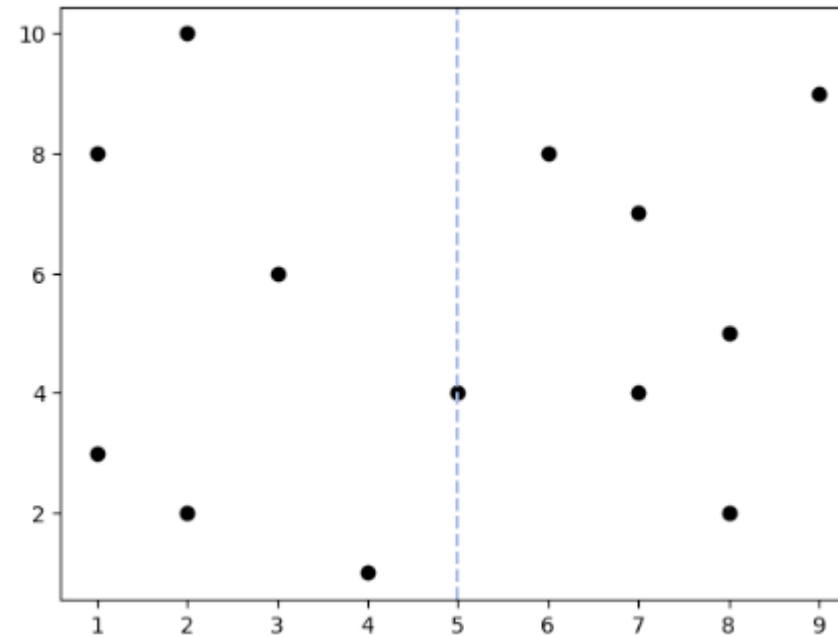
$$P_9(7, 4)$$

$$P_{10}(7, 7)$$

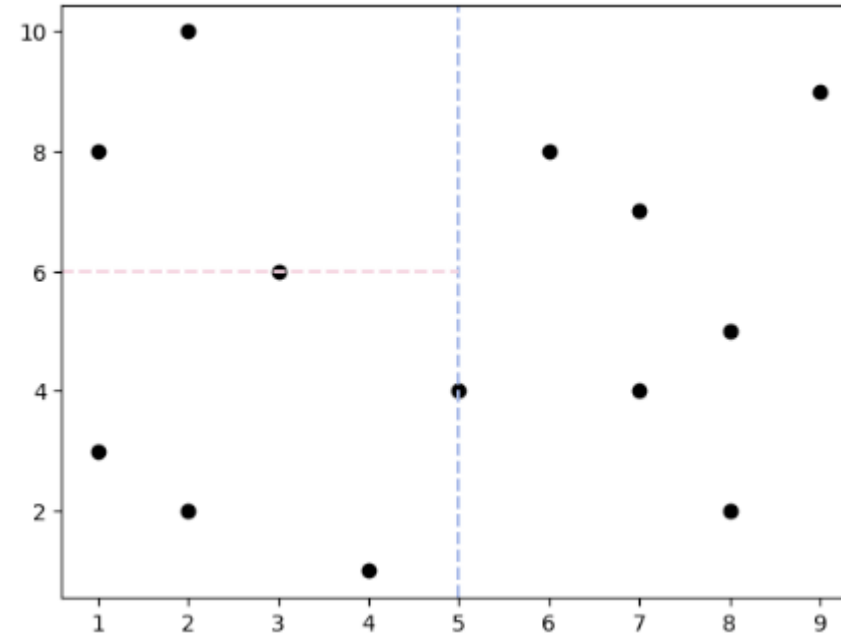
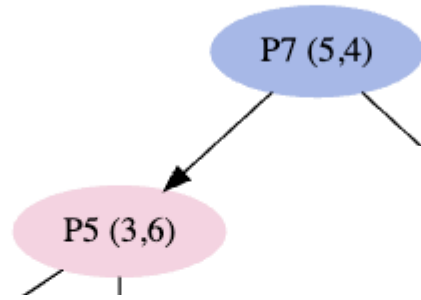
$$P_{11}(8, 2)$$

$$P_{12}(8, 5)$$

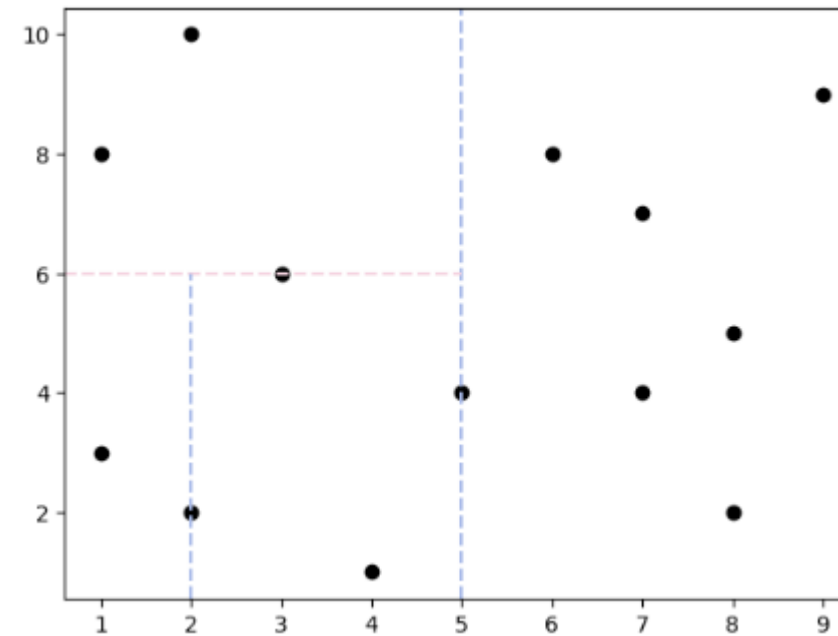
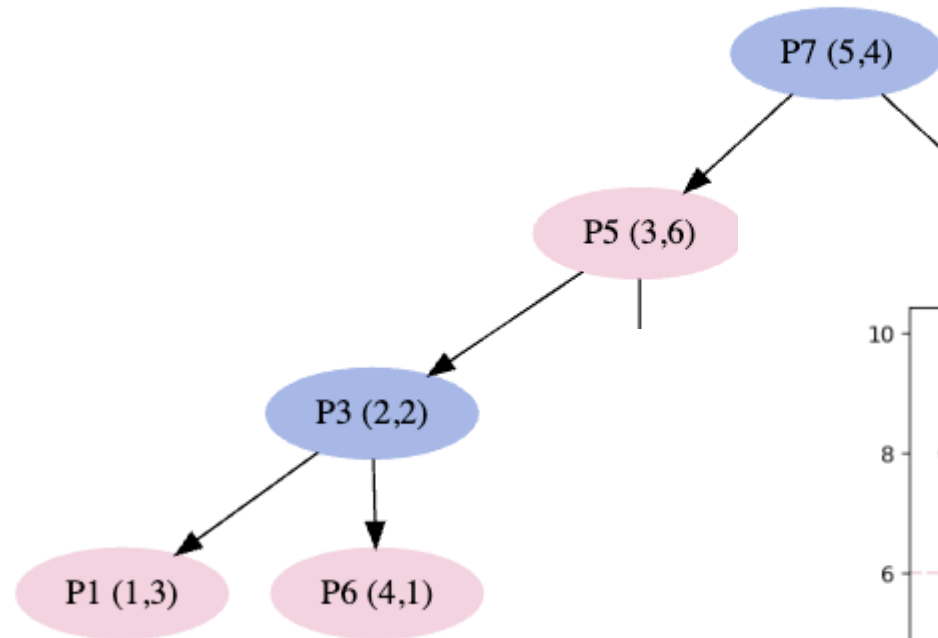
$$P_{13}(9, 9)$$



$P_1(1, 3)$
 $P_2(1, 8)$
 $P_3(2, 2)$
 $P_4(2, 10)$
 $P_5(3, 6)$
 $P_6(4, 1)$
 $P_7(5, 4)$
 $P_8(6, 8)$
 $P_9(7, 4)$
 $P_{10}(7, 7)$
 $P_{11}(8, 2)$
 $P_{12}(8, 5)$
 $P_{13}(9, 9)$



$P_1(1, 3)$
 $P_2(1, 8)$
 $P_3(2, 2)$
 $P_4(2, 10)$
 $P_5(3, 6)$
 $P_6(4, 1)$
 $P_7(5, 4)$
 $P_8(6, 8)$
 $P_9(7, 4)$
 $P_{10}(7, 7)$
 $P_{11}(8, 2)$
 $P_{12}(8, 5)$
 $P_{13}(9, 9)$



Uso de KDTree con DBSCAN

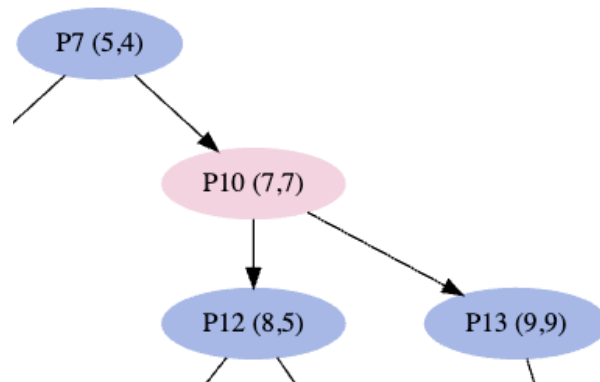
Con la estructura de árbol, a la hora de calcular la vecindad alrededor de un punto no es necesario calcular todas las distancias.

Para ello se va a los nodos cercanos al punto en el que queremos calcular las distancias.

Si la distancia es menor que el radio $\rightarrow$ continuar con todas las ramas

Si es mayor $\rightarrow$ continuar por la rama en la que está el punto

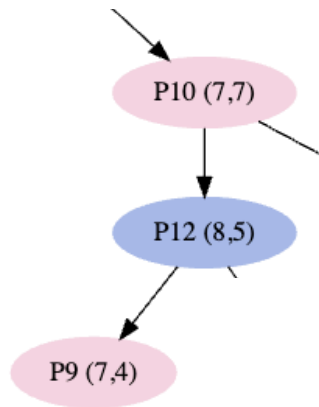
Ejemplo: cálculo de la vecindad de $P_{10}(7,7)$ con radio $\varepsilon = 2$.



$$d(P_{10}, P_7) = 3.61 > \varepsilon; \text{ Como } 7 > 5 \Rightarrow \text{derecha}$$

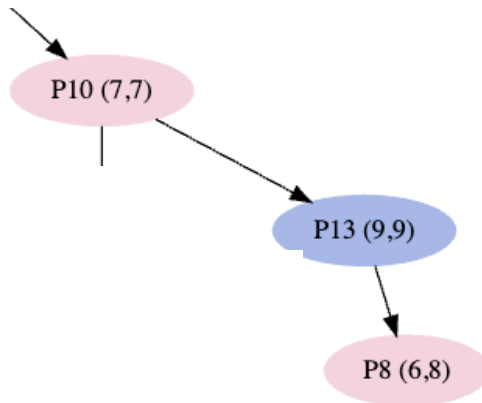
$$d(P_{10}, P_{12}) = 2.23 > \varepsilon; \text{ Como } 7 < 8 \Rightarrow \text{izquierda}$$

$$d(P_{10}, P_{13}) = 2.8 > \varepsilon; \text{ Como } 7 < 9 \Rightarrow \text{izquierda}$$



$d(P_{10}, P_{12}) = 2.23 > \varepsilon$; Como $7 < 8 \Rightarrow$ izquierda

$d(P_{10}, P_9) = 3 > \varepsilon$; Aquí el árbol ha terminado



$d(P_{10}, P_{13}) = 2.8 > \varepsilon$; Como $7 < 9 \Rightarrow$ izquierda

$d(P_{10}, P_8) = 1.41 < \varepsilon$; Aquí el árbol ha terminado

En resumen:

$$d(P_{10}, P_7) = 3.61 > \varepsilon$$

$$d(P_{10}, P_{12}) = 2.23 > \varepsilon$$

$$d(P_{10}, P_{13}) = 2.8 > \varepsilon$$

$$d(P_{10}, P_9) = 3 > \varepsilon$$

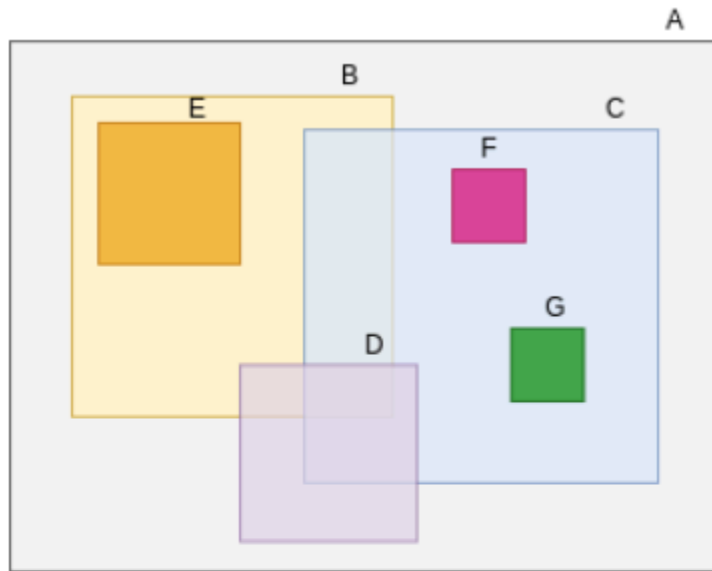
$$d(P_{10}, P_8) = 1.41 < \varepsilon$$

$$N_{\varepsilon=2}(P_{10}) = \{P_8\}$$

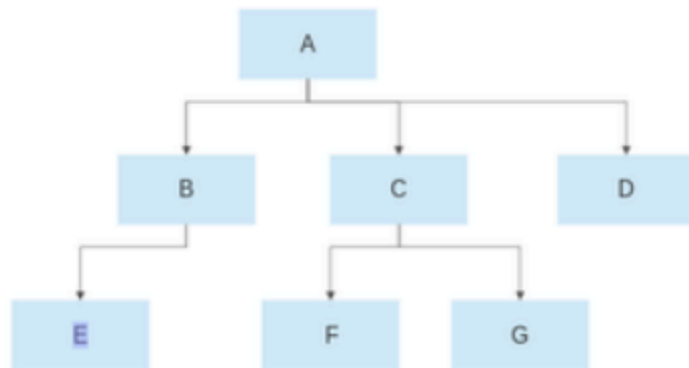
Para obtener la vecindad se han calculado 5 distancias en lugar de 12.

RTree

- Diseñados para indexar datos espaciales.
- Se particionan los datos en "cajas" o rectángulos delimitantes mínimos (MBR, Minimum Bounding Rectangle), que pueden solaparse.
- Pueden mejorar la eficiencia al permitir búsquedas rápidas de puntos dentro de un radio ε , similar a como se utilizan los KD-Trees.
- Adecuado para estructuras complejas y datos espaciales.



- Nodo raíz e intermedios: apuntan a otros nodos que tienen por debajo de menor nivel
- Nodos terminales: son MBR que apuntan directamente a los datos de origen.



Construcción:

Para la construcción de un RTree se definen M y m :

- M : Cada nodo, excepto el raíz, debe tener como máximo M entradas
- m : Cada nodo excepto el raíz, debe tener como mínimo m entradas — se suele tomar $\lceil M/2 \rceil \equiv \text{ceil}(M/2)$

Si un nodo alcanza el máximo de su capacidad, se divide y se incluye el hijo que se quiere insertar en una de las divisiones. Para ello se tiene en cuenta que:

- el área resultante sea la menor de entre las opciones
- la expansión (área previa/área nueva) sea la menor posible

Ejemplo

Sean los puntos A,...,F. Creación de un RTree con $M = 3$ y $m = 2$

$A(1, 3)$

$B(2, 2)$

$C(3, 6)$

$D(4, 1)$

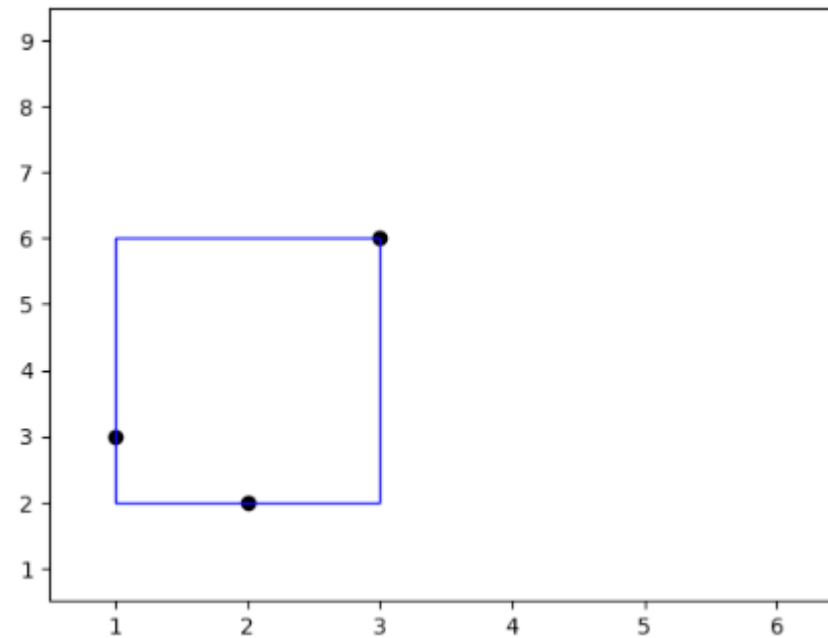
$E(5, 4)$

$F(6, 9)$

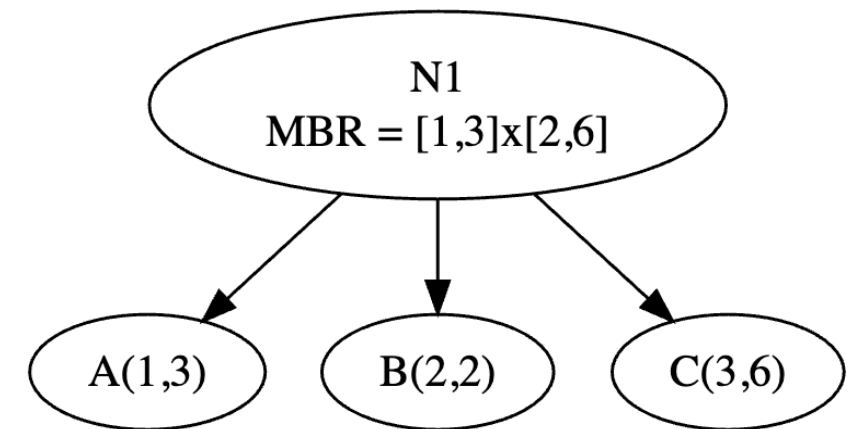
Paso 1

Crear N_1 e insertar elementos hasta llegar a M

Construir el MBR



$$N_1 = \{A, B, C\}$$
$$\text{MBR} = [1, 3] \times [2, 6]$$
$$S = 8$$



$A(1, 3)$

$B(2, 2)$

$C(3, 6)$

$D(4, 1)$

$E(5, 4)$

$F(6, 9)$

Paso 2

Al insertar D se alcanza el máximo.

Se divide en dos conjuntos usando como semillas los puntos con mayores distancias (A,D).

Opción 1: $(AB, CD) \rightarrow ([1, 2] \times [2, 3], [3, 4] \times [1, 6]) \rightarrow S = 6$

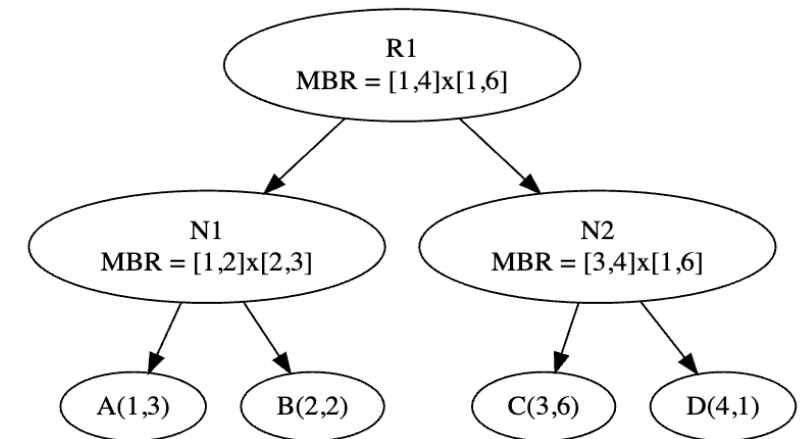
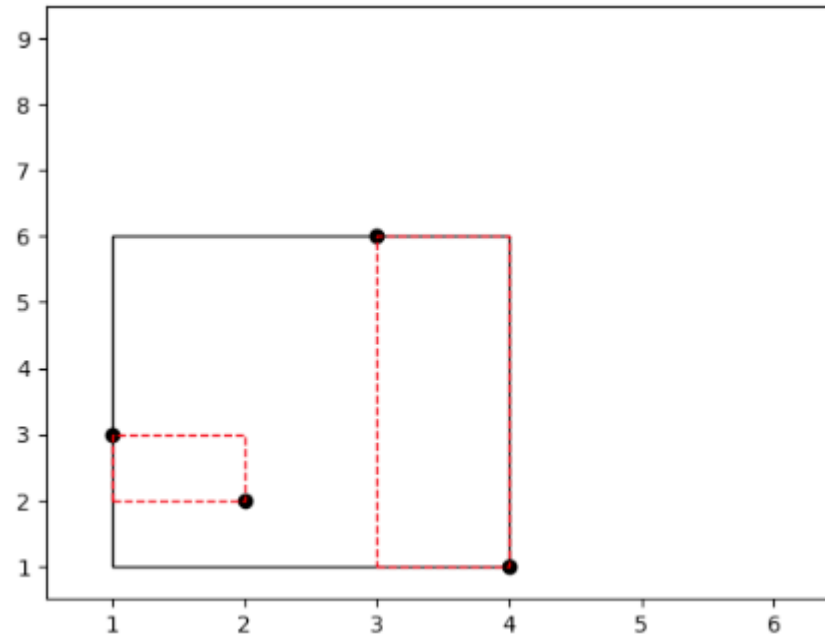
Opción 2: $(AC, BD) \rightarrow ([1, 3] \times [3, 6], [2, 4] \times [1, 2]) \rightarrow S = 8$

La opción con menor área es la primera. Se crea un nodo interno R1 que engloba a dos nuevos nodos

$N_1 = \{A, B\}$
 $MBR = [1, 2] \times [2, 3]$

$N_2 = \{C, D\}$
 $MBR = [3, 4] \times [1, 6]$

$R_1 = \{N_1, N_2\}$
 $MBR = [1, 4] \times [1, 6]$



$A(1, 3)$

$B(2, 2)$

$C(3, 6)$

$D(4, 1)$

$E(5, 4)$

$F(6, 9)$

Paso 3

Insertamos E

Se tienen dos opciones. Queremos minimizar el aumento de área

Opción 1: añadirlo a N_1

$$N_1 = \{A, B, E\} \rightarrow \text{MBR} \equiv [1, 5] \times [2, 4] \rightarrow \Delta S = 7$$

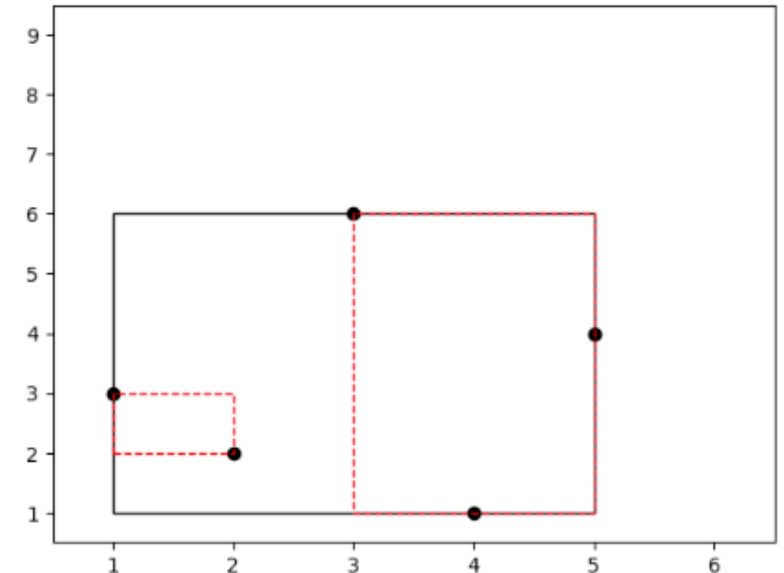
Opción 2: añadirlo a N_2

$$N_2 = \{C, D, E\} \rightarrow \text{MBR} \equiv [3, 5] \times [1, 6] \rightarrow \Delta S = 5$$

El aumento es menor si se añade al segundo nodo.

Hay que recalcular R_1

$$R_1 = \{C, D, E\} \rightarrow \text{MBR} \equiv [1, 5] \times [1, 6]$$



$A(1, 3)$

$B(2, 2)$

$C(3, 6)$

$D(4, 1)$

$E(5, 4)$

$F(6, 9)$

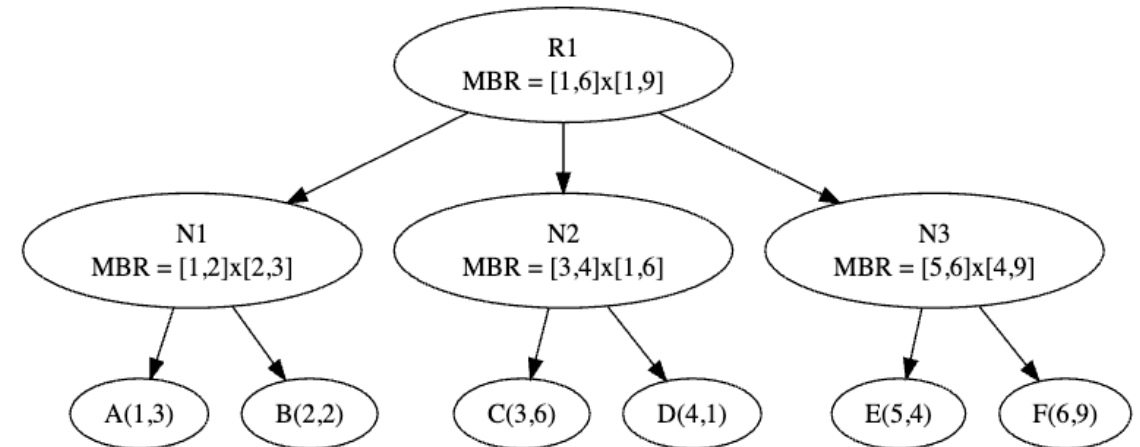
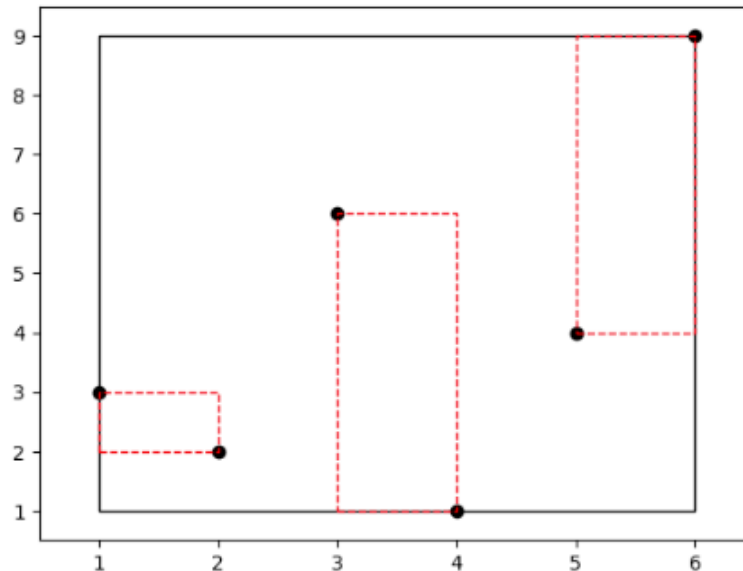
Paso 4

Insertamos F

Se tienen dos opciones.

Si se inserta en N_1 la diferencia de área sería 34 mientras que si se inserta en N_2 , la diferencia es de 19. Por tanto, se añade a N_2 .

Como N_2 ya tiene el máximo número que puede almacenar, hay que dividir (como en el Paso 2). Finalmente, queda:



Uso de RTree con DBSCAN

Se parte de una región de búsqueda (por ejemplo, un punto y un radio/epsilon, que define un círculo o un cuadrado).

Se empieza en la raíz.

Para cada punto en el nodo actual:

Se comprueba si el MBR de la entrada se solapa con la región de búsqueda.

Si hay solape:

Si el nodo actual es interno, se sigue el puntero al nodo hijo y se repite el proceso de búsqueda recursivamente en ese subárbol.

Si el nodo actual es hoja, se comprueba si el objeto real asociado a la entrada está dentro de la región de búsqueda.

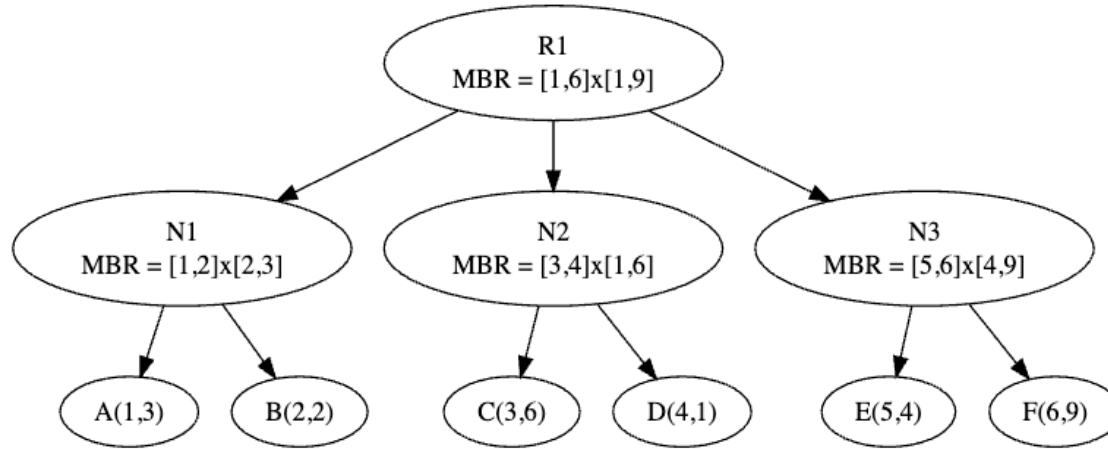
Si lo está, se añade a los resultados.

Si no hay solape:

Se ignora esa entrada y su subárbol (poda), lo que hace la búsqueda eficiente

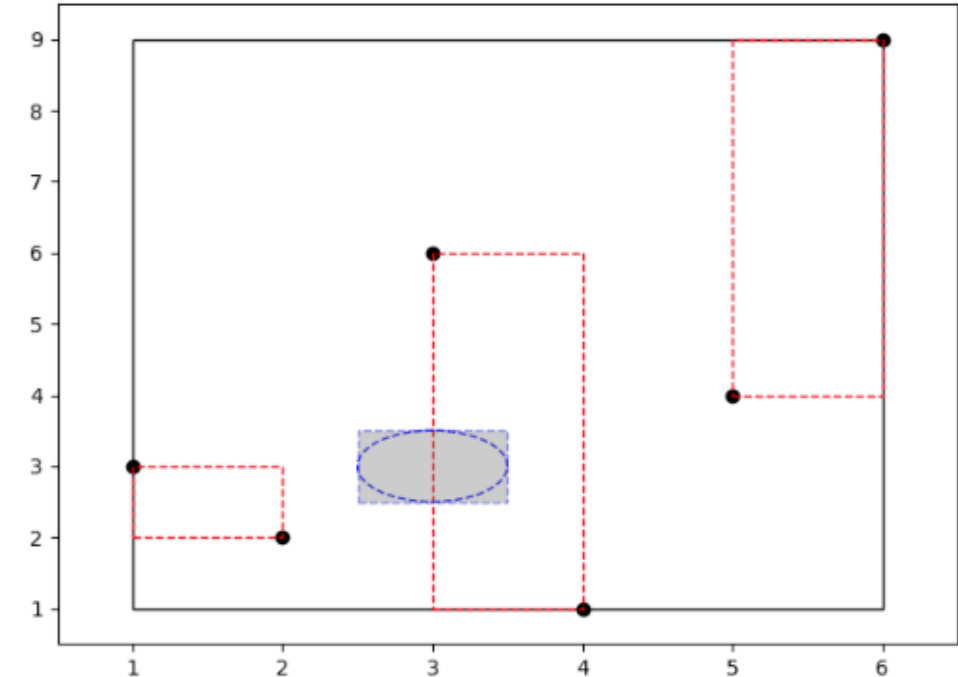
Ejemplo (ejemplo anterior). Construcción de la vecindad de radio 0.5 alrededor de $P(3,3)$

$A(1, 3)$
 $B(2, 2)$
 $C(3, 6)$
 $D(4, 1)$
 $E(5, 4)$
 $F(6, 9)$



Primero, se construye el MBR de la zona

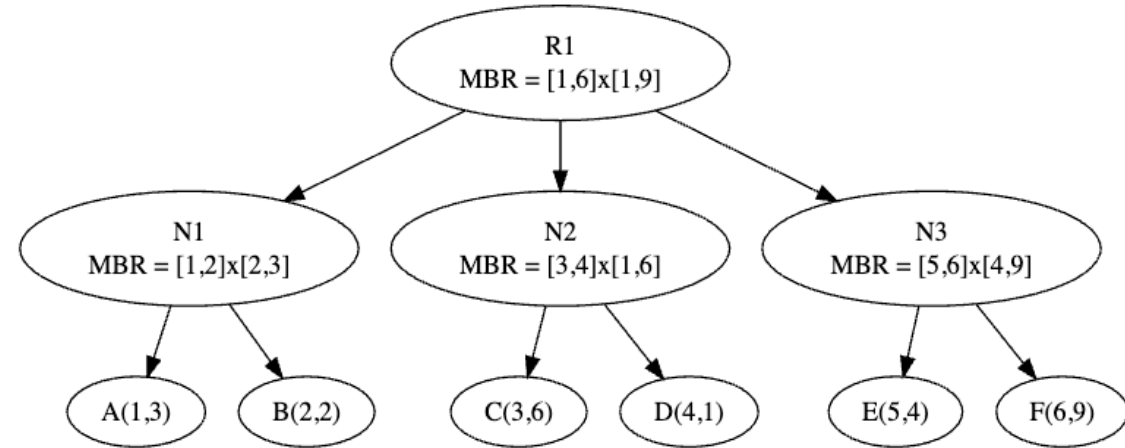
$$\begin{aligned}
 &P(3, 3) \\
 &\varepsilon = 0.5 \\
 &\text{MBR} = [2.5, 3.5] \times [2.5, 3.5]
 \end{aligned}$$



Paso 1

Mirar si en el nodo raíz hay R_1 solape.

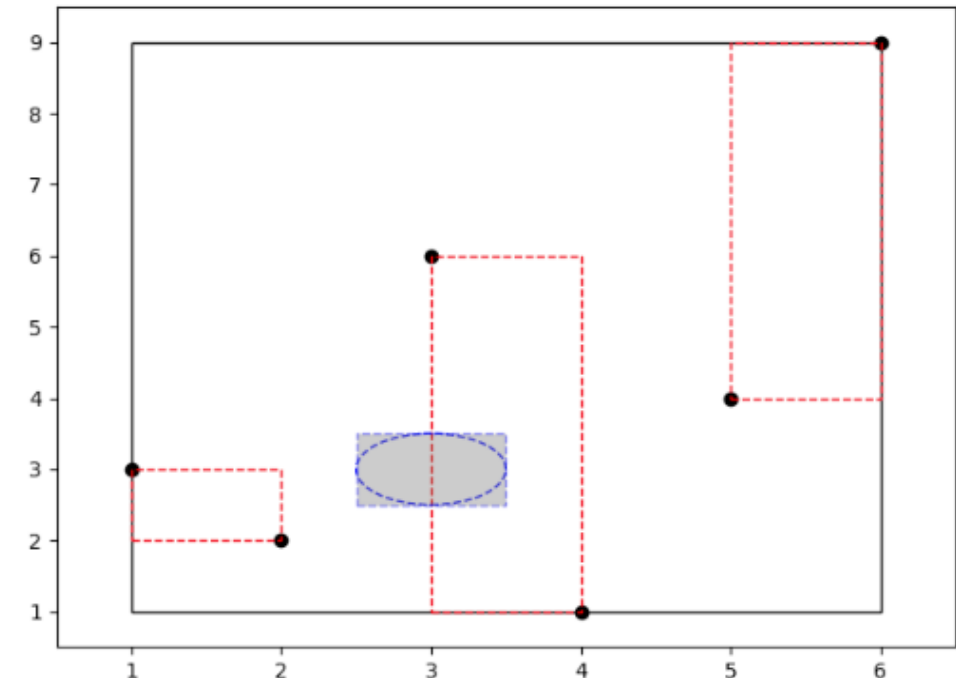
Como sí lo hay y no es un nodo terminal miramos sus hijos.



Paso 2

Mirar si hay solape para

- N_1 : no lo hay -> eliminar subárbol
- N_2 : sí lo hay -> como es un nodo terminal, calcular distancias con los puntos en el Paso 3
- N_3 : no lo hay -> eliminar subárbol



Paso 3

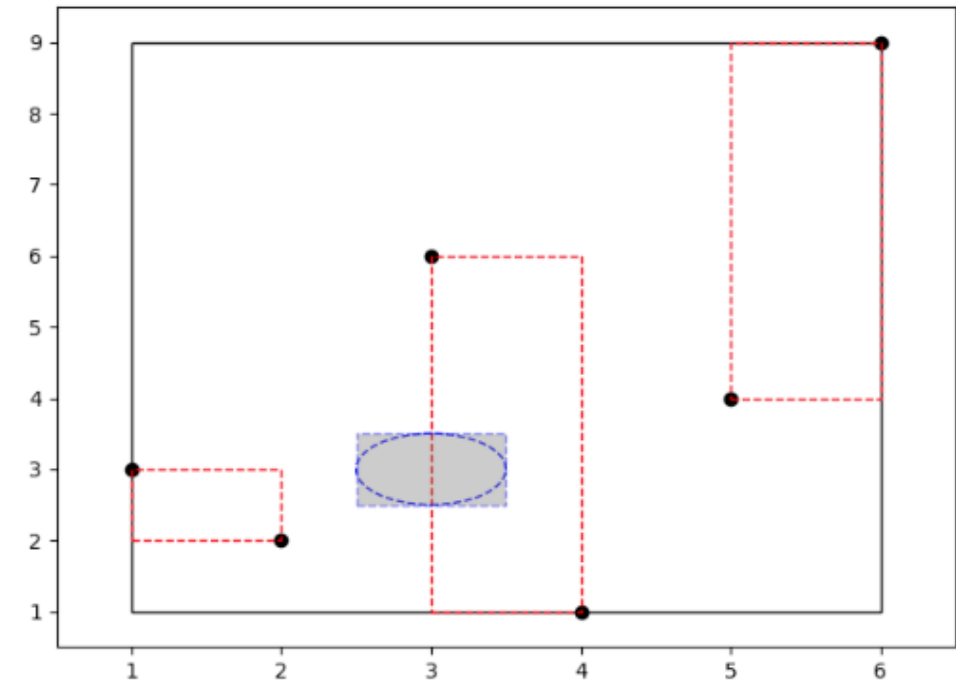
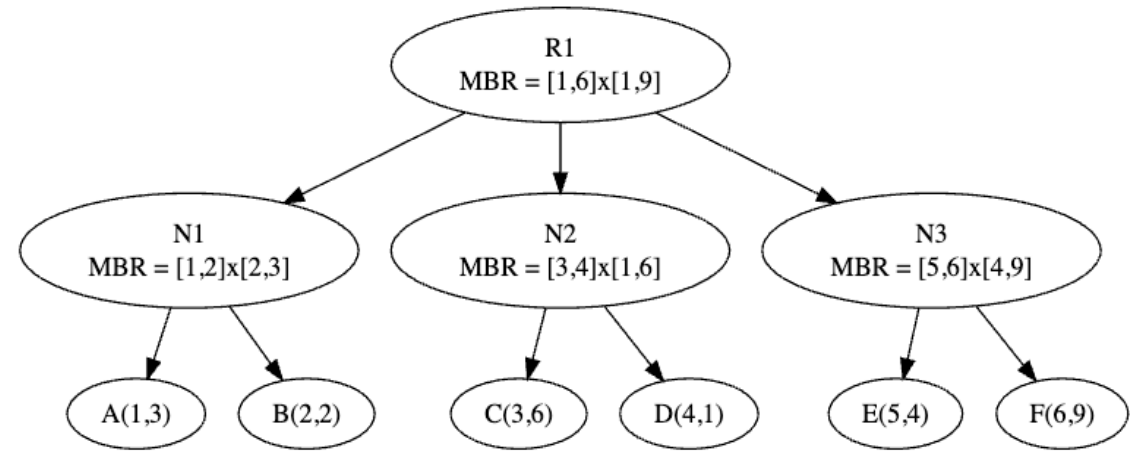
Calcular las distancias a los puntos

$$d(P, (3,6)) > \varepsilon$$

$$d(P, (1,4)) > \varepsilon$$

En este caso, no hay ningún vecino en el radio especificado.

$$N_{\varepsilon=0.5}(P) = \emptyset$$





www.unir.net